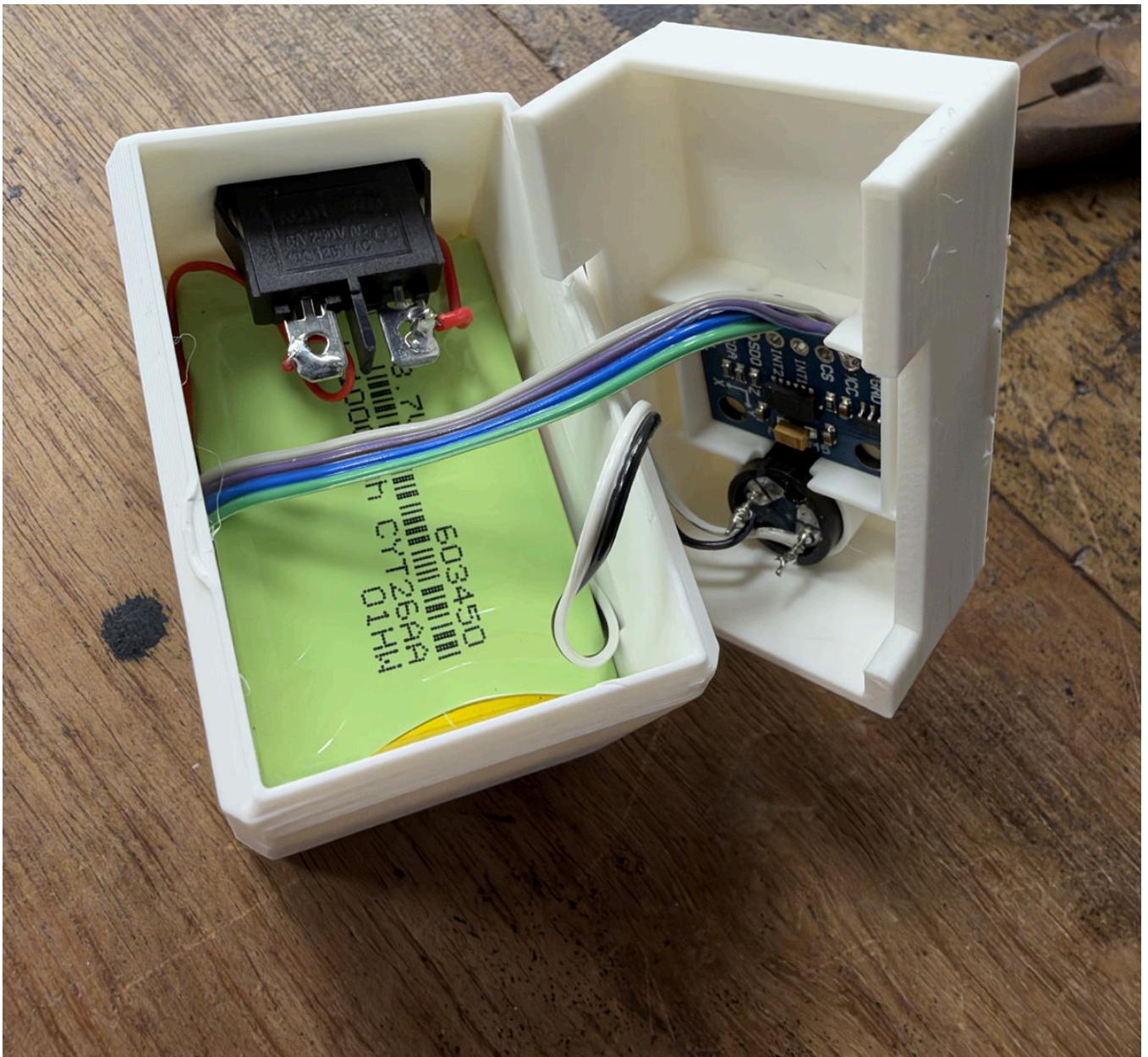


自製計步器

從硬體、演算法到模擬器

TTGO T-Display × ADXL345 × 蜂鳴器 | 中文版韌體與瀏覽器模擬器 | 教學講義



目錄

這份講義怎麼讀：每一節先看「它做了什麼」，再講「為什麼」，最後是「所以我們要注意什麼」。

- 1 先看成果：這台裝置會做什麼
 - 2 硬體解剖：每個零件在做什麼
 - 3 接線與原因：為什麼接這幾支腳
 - 4 從加速度到步數：演算法怎麼想
 - 5 模組比較與選型：換一顆行不行
 - 6 軟體架構與模擬器：先在電腦上跑
 - 7 失敗路徑：我們踩過的坑
 - 8 兩個逆向案例：Pico 成功、ESP32 停下
 - 9 延伸專案（七個，程式在模擬器網站）
 - 10 遊戲化方向：卡帶架構、擲筊 / 骰子已實作、A / B 兩條路線
- 附錄 腳位表、API、指令、材料包、官方資料、除錯流程、審稿紀錄、圖片出處

這份講義的來源

原作是學生專題「GOTT Pedometer」（英文介面）。學生電腦故障、原始碼遺失，我們先把裝置的快閃記憶體做了唯讀備份與分析，再以原作的功能為規格重新設計中文版，並做出一個在瀏覽器裡就能跑的模擬器。第 8 節會說明「為什麼原始碼抓不回來」。

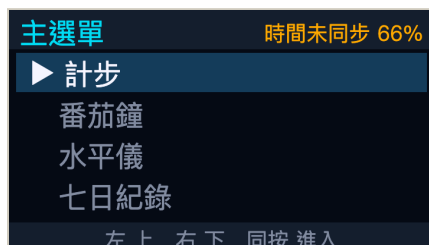
需要準備

硬體：LILYGO TTGO T-Display V1.1、ADXL345 (GY-291) 模組、無源蜂鳴器、3.7 V 鋰電池（1000 mAh，含開關）、杜邦線、USB-A 轉 USB-C 線。

軟體：任何能開網頁的電腦（模擬器）；要燒錄時需 Arduino ESP32 核心 3.3.x（本專案用 `arduino-cli`）。

1 先看成果：這台裝置會做什麼

開機後是一個六項的中文主選單（第五項「遊戲」見第 10 節）。兩顆板載按鍵就是全部的操作：左鍵「上 / 加 / 存檔」、右鍵「下 / 減 / 開始暫停」、兩鍵同按短放「進入 / 確認」、兩鍵同按長按「回主選單」。



主選單：計步、番茄鐘、水平儀、七日紀錄、無線網路。右上角顯示時間（未同步時提示）與電量。



計步：步數、每日目標進度條、估算卡路里與距離。找不到加速度計時會直接寫在畫面上。



番茄鐘：工作 / 休息時段倒數，結束時蜂鳴器播放旋律，累計完成顆數。

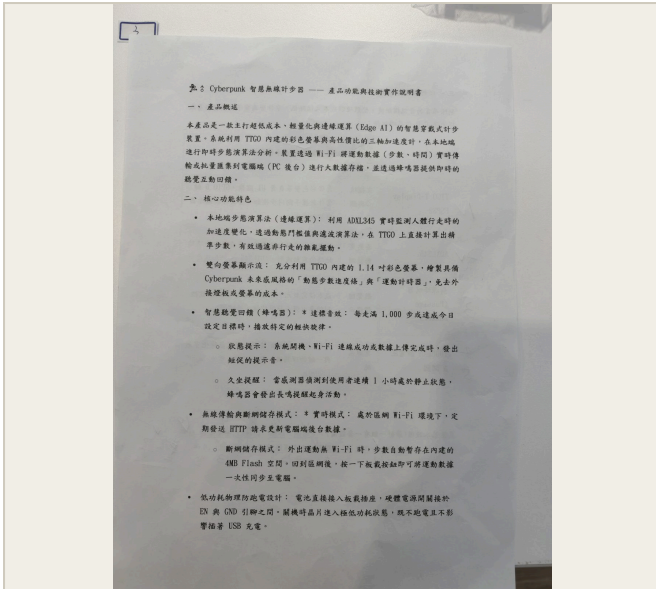
功能	做什麼	原作有嗎 / 新版補了什麼
計步	以 ADXL345 的加速度判斷步伐，算步數、距離（步幅）、卡路里（體重）	原作有。新版把步幅、體重、目標、演算法參數都做成可調
番茄鐘	工作 25 分 / 休息 5 分（可調），結束響鈴	原作有。新版加旋律、跳過、網頁遙控
水平儀	用重力方向算俯仰（pitch）與橫滾（roll）角， $\pm 2^\circ$ 內算水平	原作有
七日紀錄	每天的步數與番茄鐘數	原作有欄位但沒有時鐘、不會換日。新版由網頁同步時間後真正能換日
無線網路與網頁	裝置自己開熱點；螢幕顯示 Wi-Fi QR Code，手機相機掃一下就加入，連上後強制門戶讓儀表板自動跳出（不必打網址）	原作有，英文、無密碼、要手動輸入 SSID 與網址。新版中文、掃碼即連、自動跳頁、會改資料的操作才要密碼、顯示電量
待機時鐘	30 秒沒按鍵就進入「上面大字時刻、下面一條會動的關卡」畫面（參考 Game & Watch 時鐘的結構）：小跑者在磚地上跑，走路時場景加速捲動、靜止時慢慢散步；右邊會掉刺球、自動跳過；每整點撞問號方塊噴三枚金幣 + 金幣音，數字變金色；每滿 100 步噴一枚金幣。背光降到 30%，計步照跑；任一鍵回選單	原作無。時間來自手機開網頁或電腦序列埠的同步

設定選單	無線網路 (QR)、個人資料 (性別 / 年齡 / 身高 / 體重 / 步幅)、蜂鳴器音量 (關 / 小 / 中 / 大)、螢幕亮度 (30 / 60 / 100%)、操作說明、關於、關機	原作無。卡路里改依個人資料計算，見 4.4
關機	設定→「關機」→ 同按確認 → 存檔、時間寫入 RTC → 深度睡眠；右鍵開機，時間接回	原作是剪電池紅線串實體開關。新版不裝開關、不剪電池線 (第 2.2 節)

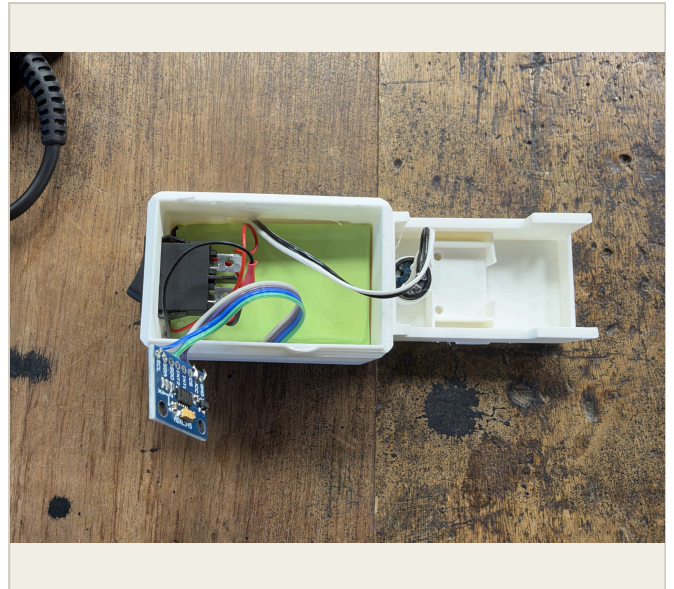
1.1 學生的說明書寫了什麼、韌體實際做了什麼

原作附有一份「Cyberpunk 智慧無線計步器——產品功能與技術實作說明書」。把它和從韌體裡取回的字串、分區、NVS 內容對照，會發現文件與實作已經「漂移」——這在真實專案裡非常常見，也是這份講義以**韌體**而非說明書為規格的原因。

說明書寫的	韌體裡找到的	怎麼解讀
動態門檻 + 濾波的本地步態演算法	有計步與自動存檔；參數細節在機器碼裡看不到	一致，但無法驗證「動態門檻」是否真的實作
Cyberpunk 風格步數進度條與運動計時器	有 <code>step_goal</code> 進度與番茄鐘 (WORK / BREAK)；畫面文字全英文	「運動計時器」做成了番茄鐘
每 1,000 步或達標播放旋律；開機 / 連線 / 上傳提示音	有 <code>toneTask</code> ；沒有任何「上傳」相關字串	有響鈴，但沒有上傳可提示
久坐 1 小時長鳴提醒	字串中完全沒有相關訊息	很可能沒做，或做了沒留訊息
實時模式：定期 HTTP 請求上傳到電腦後台	方向相反：裝置自己開熱點當伺服器，瀏覽器來連；沒有任何外部網址	架構在開發中被改掉了，文件沒更新
斷網時暫存 4 MB Flash，按板載按鈕一次同步到電腦	步數存在 NVS，有七日欄位 <code>h_step_</code> / <code>h_pomo_</code> （但沒有時鐘所以沒資料）；沒有同步功能	「存」有做，「同步」沒做
電源開關接在 EN 與 GND 之間	實物照片：開關串在電池正極線上	兩種接法各有取捨，見 2.1
(沒寫)	水平儀、七日紀錄畫面、網頁儀表板	做了沒寫進文件



原作說明書第一頁。寫作時間早於最終韌體，所以有多處與實作不符。



原作拆開後的全部零件：T-Display、ADXL345（GY-291）、無源蜂鳴器、1000 mAh 鋰電池、船型開關、3D 列印外殼。

教學點

說明書是「打算做什麼」，韌體是「真的做了什麼」。交件時要回頭用實作校正文件；拿到別人的專案時，先信實物與程式，再信文件。



無線網路畫面：右側是標準 Wi-Fi 設定 QR（WIFI:T:WPA;S:;;P:;;），iPhone / Android 相機掃到就問要不要加入。連上後手機偵測到「沒有網際網路」，裝置的 DNS 把所有網域答成自己、探測網址一律轉向儀表板，手機就自動彈出登入頁——跟飯店 Wi-Fi 一樣的機制。



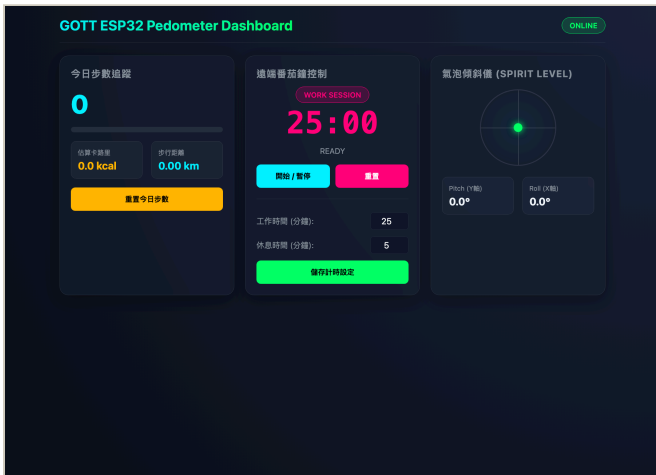
待機時鐘：上面 7 段大字（只用矩形畫），下面是會捲動的關卡——問號方塊、水管、雲、磚地與原創的小跑者（12×16 像素圖放大 2 倍）。結構參考 Game & Watch 時鐘，角色與場景皆自繪。



主選單「關機」的確認畫面：同按確認後存檔、時間寫進 RTC、深度睡眠；右鍵開機。



(第 10 節) 擲筊卡帶。



原作的網頁儀表板（從韌體裡取回的原始 HTML）：英文介面、深色卡片。



新版儀表板在模擬器裡執行：狀態、番茄鐘、水平儀、七日紀錄、設定五個分頁，全中文。

2 硬體解剖：每個零件在做什麼

零件	角色	關鍵規格（為什麼重要）
TTGO T-Display 主控 + 螢幕	ESP32 雙核微控制器，板上就有 1.14 吋彩色螢幕、兩顆按鍵、USB 轉序列晶片、鋰電池充電與電池座	3.3 V 邏輯；4 MB 快閃記憶體（程式分區只有 1.2 MB，第 7 節會踩到）；Wi-Fi 開啟時耗電 80~240 mA
ADXL345 三軸加速度計	量三個方向的加速度，靜止時量到的是重力（1 g）。走路的上下震動、傾斜時重力方向的改變，都靠它	±2~16 g 可選；每格 3.9 mg；I2C 位址 0x53；GY-291 模組板上有穩壓器與上拉電阻
無源蜂鳴器	番茄鐘結束、按鍵回饋、達成目標時發聲	「無源」表示內部沒有振盪器，要由 GPIO 送方波（ <code>tone()</code> ）才會響，所以能選音高、放旋律
鋰電池 1000 mAh + 電源開關	離開 USB 也能用。開關串在正極線上	3.7 V 標稱、4.2 V 滿電、3.3 V 以下應停用；接 T-Display 的 JST 電池座，板上 TP4054 會在插 USB 時充電

鋰電池安全守則（實作前先讀）

① 不要同時剪紅黑兩條線：斜口鉗刀口會讓正負極瞬間短路。一次只處理一條：剝皮→套熱縮管→焊→烘縮→確認絕緣，再處理另一條。② 開關的裸露接腳一定要用熱縮管包好，塞進外殼時才不會刮破電池的鋁塑膜（照片裡原作的開關焊點就是裸露的，是要改進的地方）。③ 電壓不要低於 3.0 V；本韌體把 3.3 V 顯示為 0%，看到 0% 就關機充電。④ 電池鼓起、發熱、外膜破損一律停用。⑤ 充電時人要在場。

2.1 為什麼電池接電池座，不接 5V 腳？

- 是什麼** T-Display 的 JST 座後面接著充電晶片與電源切換電路；5V 腳是 USB 進來的 5 V。
- 為什麼** 鋰電池只有 3.7 V，直接餵 5V 腳既不夠也會繞過充電與保護；接電池座才會「插 USB 充電、拔 USB 用電池」。
- 所以呢** 開關一定串在正極線上，而且電池的負極要接到電池座的負極，不要另外拉到板子其他 GND（會形成不必要的迴路）。

開關接法	關機時發生什麼	優點	代價
串在電池正極（原作實物、本講義）	電池與板子完全斷開	真正零耗電；接線直覺	關機狀態下插 USB 不會充電（充電晶片接不到電池）
接在 EN 與 GND 之間（原作說明書）	只有 ESP32 被壓在重置狀態（約 1 μ A）；電池仍接著整塊板子與所有周邊	關機也能充電；只要兩條細線；不必剪電池線	板上穩壓器、GY-291 模組的穩壓器與電源指示 LED（約 1~2 mA）、仍在量測的 ADXL345（約 0.14 mA）都繼續耗電，合計約 2~3 mA $\rightarrow$ 1000 mAh 大約兩三週放完；螢幕是否熄滅要實測
軟體關機（本版採用）：選單「關機」 $\rightarrow$ 深度睡眠，右鍵喚醒	ESP32 約 10 μ A；程式先讓螢幕 Sleep In、ADXL345 待機、分壓關閉；板上穩壓器約 55 μ A 仍在	不裝開關、不碰電池線；關機能充電；時鐘由 RTC 繼續走，醒來接回	GY-291 的 LED（1~2 mA）仍亮，除非拆掉；程式當掉時只能拔電池；喚醒 = 重開機約 1~2 秒

本版的決定：像 Flipper Zero 一樣，不裝實體開關

原作實物是剪斷電池紅線串開關（能動，但剪線那一刀與裸露焊點是鋰電池最大的風險）。新版改成：待機 30 秒進時鐘畫面（有走路小人動畫、計步照跑）；主選單最後一項「關機」確認後進深度睡眠，時間寫進 ESP32 的 RTC 記憶體繼續計時，按右鍵開機後接回。電池紅黑線**直接插 JST 座、不剪**。用 GPIO35 而不是 GPIO0 喚醒，是因為 GPIO0 在開機瞬間被按著會進燒錄模式。想要「真正零耗電」的替代方案：做一條 JST 轉接線把開關串在轉接線上，原廠電池線一樣不剪。

開關接在 EN—GND：焊在 RST 按鍵的焊點上（板子正面朝上）

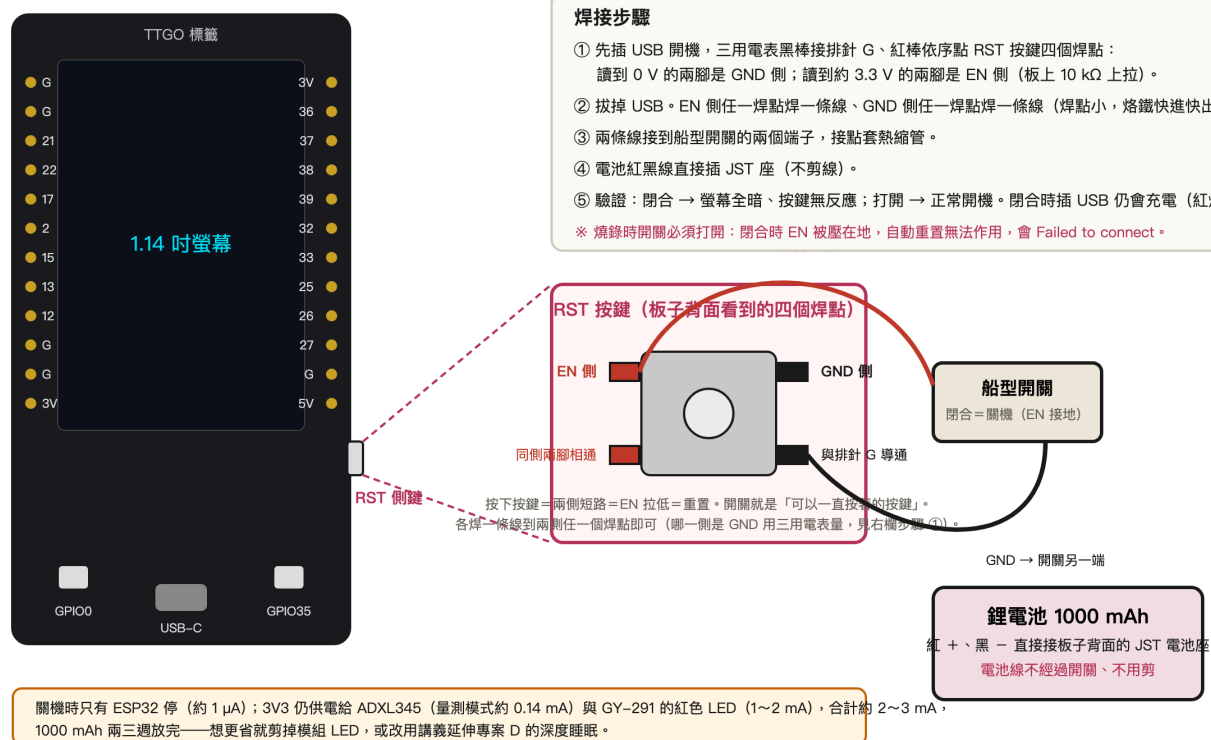


圖 2-1 若選 EN—GND 接法：EN（CHIP_PU）沒有拉到排針，只能焊在側邊 RST 按鍵的焊點上；哪一側是 GND 用三用電表量。電池直接插 JST 座、不剪線。燒錄時開關必須打開。

說明書寫「關機時晶片進入極低功耗狀態，既不跑電且不影響插著 USB 充電」：「晶片」那半句對，「不跑電」不對——跑電的不是晶片，是板子上其他零件。這正是用三用電表串進電池線實測的好題目。模擬器「接線」分頁有兩顆示範鈕，兩種接法都可以接接看，檢查結果會寫出各自的取捨。

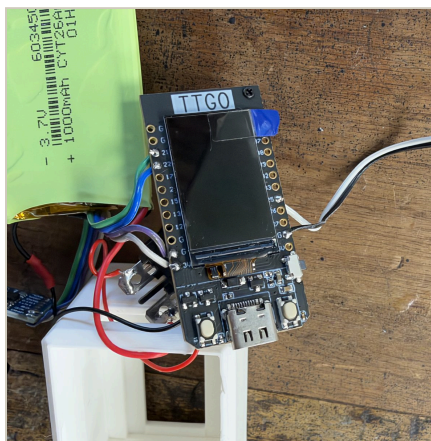
2.2 有源蜂鳴器與無源蜂鳴器

	有源（Active）	無源（Passive）←本專案
內部	有振盪器，給直流就響	只有壓電片或線圈，要外部送方波
程式	<code>digitalWrite(HIGH)</code>	<code>tone(pin, 頻率, 時間)</code>
能做的事	單一音高的嗶聲	任意音高、旋律、音效
怎麼分辨	外觀很像。背面被黑膠封住、有「+」記號的多半是有源；看得到綠色電路板或線圈的多半是無源。最保險：接 3.3 V 直流，會響的是有源。	
接反會怎樣	有源有極性，接反不響	無源沒有嚴格極性，接反照樣響；習慣上「+」接 GPIO

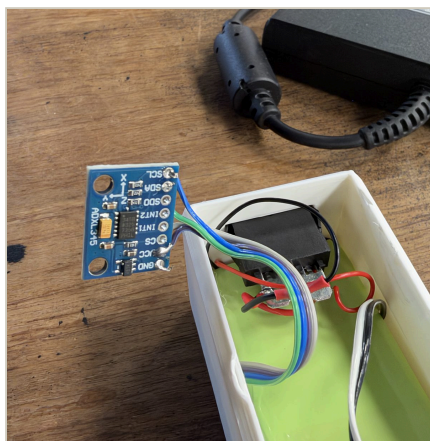
常見誤判

學生把有源蜂鳴器接上 `tone()`，結果只有「沙沙聲」或固定音高——因為內部振盪器會蓋過外部方波。反過來把無源蜂鳴器接直流，則完全不響。

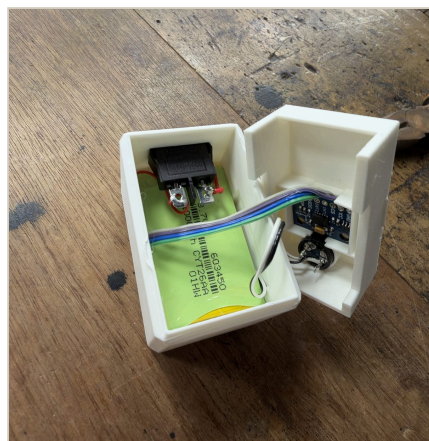
2.3 實物長這樣



T-Display 背面：排線焊在 21、22、G、3V；蜂鳴器白線焊 25、黑線焊 G；電池紅黑線進 JST 座。



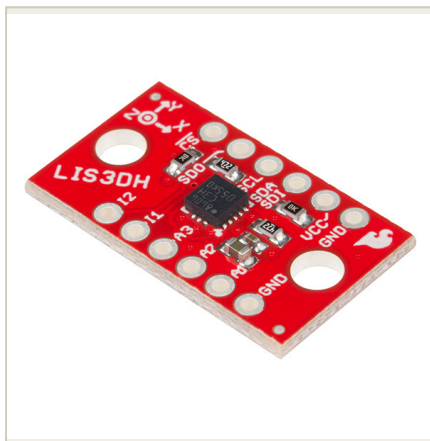
GY-291 ADXL345：只焊 SCL（藍）、SDA（綠）、VCC、GND 四支腳；CS、SDO、INT1、INT2 不接。



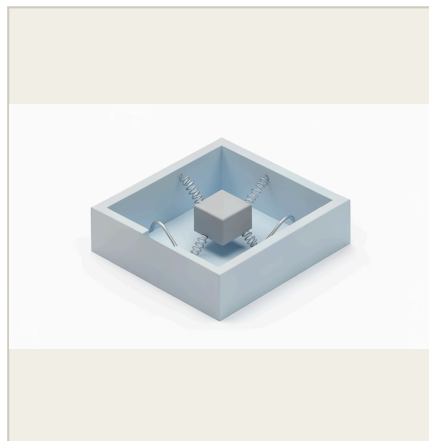
外殼下半部放船型開關與電池，上蓋放 ADXL345 與黑色圓殼的無源蜂鳴器，排線穿過側邊缺口。



替代型號 MPU6050 (GY-521)：加速度+陀螺儀，位址 0x68。



替代型號 LIS3DH (SparkFun 版)：超省電，穿戴裝置常用。



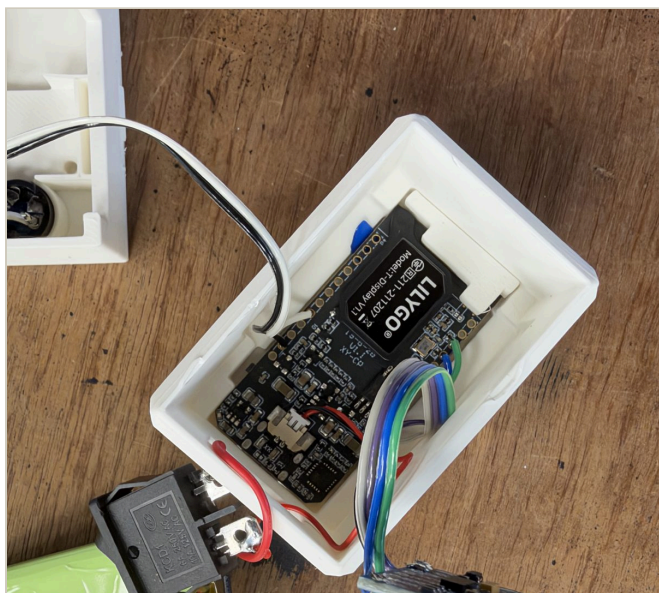
MEMS 加速度計原理示意：一塊微小的質量塊用彈簧吊在框架裡，加速時質量塊相對框架位移，位移量就對應加速度。實際晶片裡的「彈簧」是蝕刻出的摺疊矽樑，位移靠梳狀電極的電容變化量出來。(AI 生成示意圖，非實際晶片結構)

2.4 T-Display 上你會用到的腳位

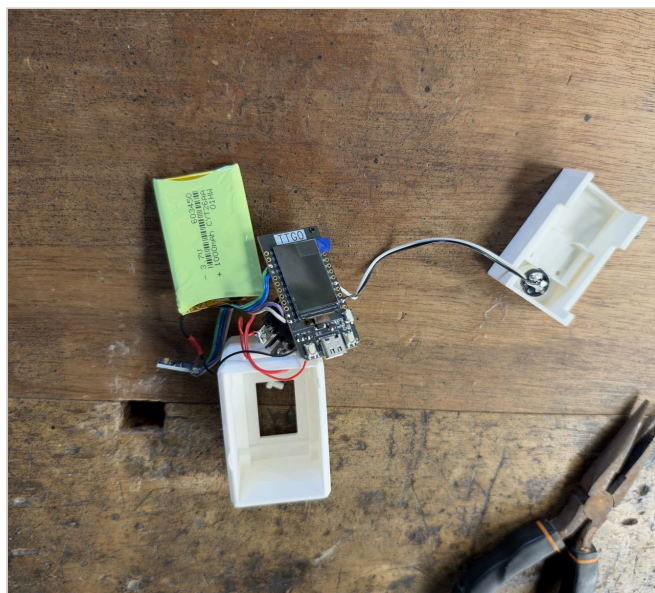
腳位	用途	要注意
GPIO0	左鍵	也是「開機模式」腳：開機瞬間按著會進入燒錄模式，所以不要讓程式在開機時依賴它

GPIO35	右鍵	只能輸入、 沒有內部上拉 ，板子上有外接上拉電阻；自己接按鍵到 34~39 時要記得加電阻
GPIO21 / 22	I2C 的 SDA / SCL	ESP32 的 I2C 腳位可以指定，21 / 22 是 Arduino 預設；ADXL345 接這兩支
GPIO25	蜂鳴器	任何可輸出的 GPIO 都行；25 同時是 DAC 腳，之後想播放聲音檔也方便
GPIO34	電池電壓	板上分壓 1/2，程式讀到的電壓要 $\times 2$ ；只能輸入
GPIO14	ADC_EN（電池量測開關）	官方腳位圖：要先把 GPIO14 拉高，GPIO34 才量得到電池電壓。我們第一版漏了這支腳，看官方資料才補上（第 7 節）
GPIO4、5、16、18、19、23	板載螢幕（背光、CS、DC、SCLK、MOSI、RST）	已經被螢幕用掉，不要再接別的東西

2.5 原作的外殼與組裝



T-Display 放進外殼下半部的凹槽，USB-C 朝外；電池紅黑線接 JST 座。



攤開：主板、電池、開關、ADXL345、蜂鳴器與兩件式 3D 列印外殼。排線用四芯彩虹線，剪下後直接焊在板子的通孔上。

是什麼

兩件式 3D 列印外殼：下半部放主板、電池與開關，上蓋放感測器與蜂鳴器；感測器用四芯排線接到主板。

為什麼

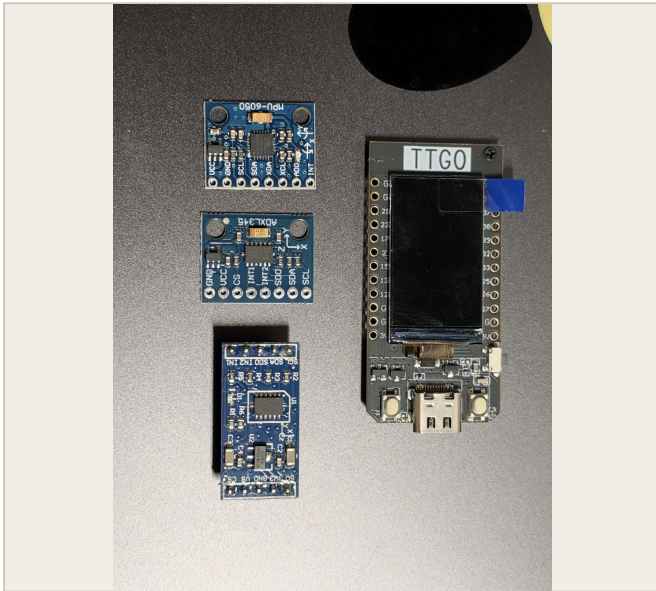
感測器放上蓋、固定在殼上，走路時的震動才會完整傳到晶片；放在鬆動的電池旁會多出很多雜訊。蜂鳴器靠近蓋子出音孔聲音才大。

所以呢

排線長度要留夠開蓋，但不能壓到電池邊緣（鋰電池被尖物壓到有危險）；開關的焊點用熱縮套管包起來，避免碰到電池鋁箔。新版韌體不改任何接線，直接燒進這個殼就能用。

2.6 實作：剝線、沾錫、焊接（新版實作紀錄，2026-10-07）

新版的基礎組合只有四樣：ADXL345、無源蜂鳴器、鋰電池（直接插 JST）、主板。接線總共 6 條：ADXL345 四條（3V3 / GND / SDA / SCL）、蜂鳴器兩條（GPIO25 / GND）。下面是實際做一遍的紀錄。

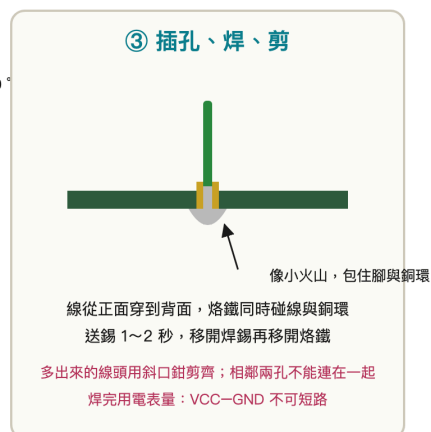


手邊的三塊感測板與主板。上：MPU-6050（GY-521，8 腳）；中：ADXL345（GY-291，8 腳橫排）；下：另一款 ADXL345 模組（腳位分兩排，同時有 5V 與 3V3 腳，接哪一支都可以，但 I2C 仍是 3.3 V）。腳位名稱相同，接法一樣。



自動剝線鉗與彩虹排線。排線先撕成四芯一組（感測器）、兩芯一組（蜂鳴器），長度約 10 cm，留開蓋的餘裕。

排線焊到模組通孔：剝、沾、焊



順序：蜂鳴器（2 線）→ ADXL345（4 線）→ 主板（6 線）→ 電池直接插 JST。每焊完一個模組就量一次。

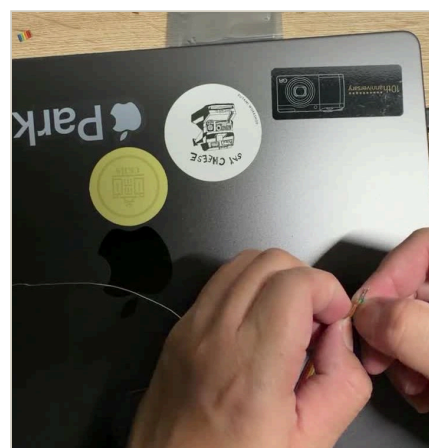
圖 2-2 每一條線都走同樣三步：剝 3 mm → 線頭沾錫 → 插孔焊接再剪齊。順序是蜂鳴器 → ADXL345 → 主板，每焊完一個模組就用電表量 VCC—GND 有沒有短路。



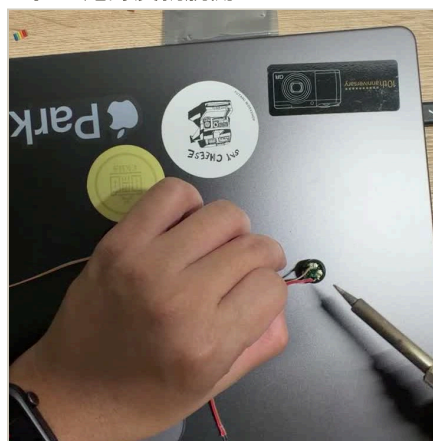
① 剝線：單線放進剝線鉗齒口，一夾一拉，絕緣皮就脫開。



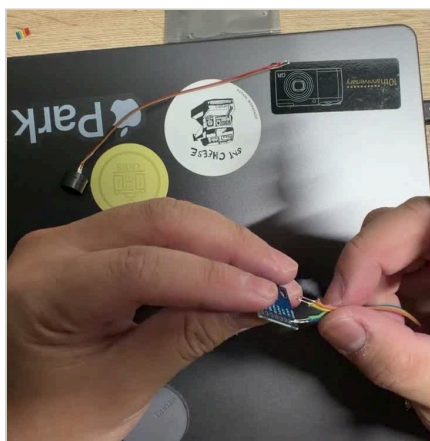
四芯都剝好、長度一致（約 3 mm）。



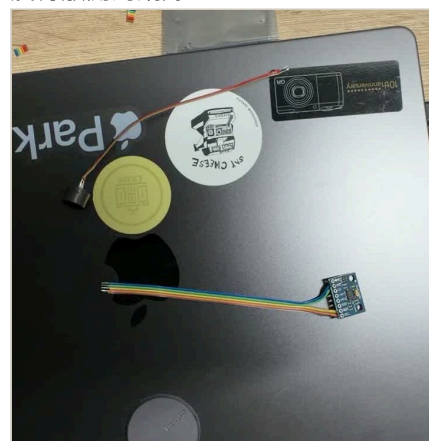
② 沾錫後線頭變成硬棒，用手指確認沒有散開的銅絲。



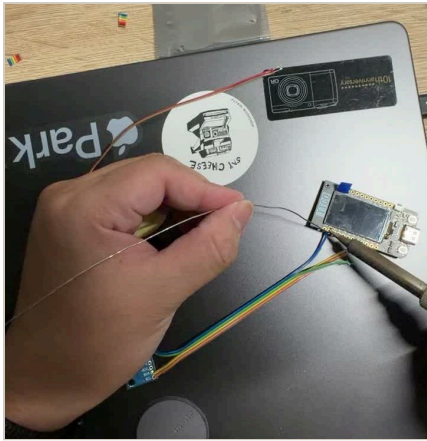
③ 先焊最簡單的蜂鳴器：兩支腳各一條線，烙鐵同時碰腳與線。



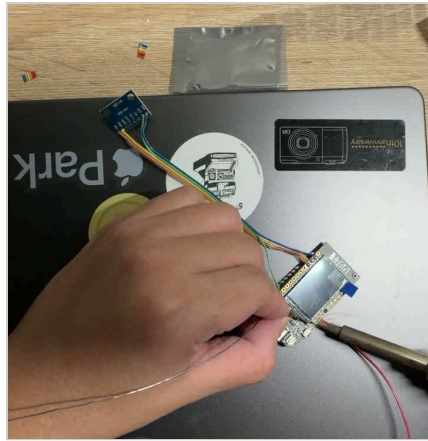
④ ADXL345：四條線穿過 VCC / GND / SDA / SCL 的通孔從背面焊，其他四孔空著。



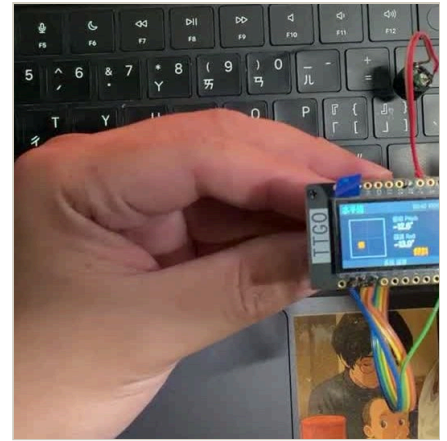
模組端完成，排線顏色對應腳位要記下來（例如 綠 SDA、藍 SCL）。



⑤ 主板端：3V、G、21、22 與 25、G 共六孔，排針孔比模組大，容易焊。



⑥ 全部焊好：主板一排線—ADXL345，蜂鳴器另一組兩芯線。



⑦ 插 USB 開機，序列埠印出「[ADXL345] 初始化成功」，螢幕進時鐘畫面。

做完的驗收

序列埠開機訊息看到 [ADXL345] 初始化成功；`status` 指令回傳 `accel_ok: true`，俯仰 / 橫滾值會跟著手上的動作變；按任一鍵有「嗶」聲。這一組接上之後，計步、水平儀、擲筊、骰子全部啟用。

實作時的提醒

① 焊接期間模組還沒裝進殼，排線很容易在焊點處折到——焊完先套熱縮管再整線。② ADXL345 的安裝方向決定水平儀的「0°」：模組要平貼上蓋、晶片面朝上；若側立，水平儀會一直顯示傾斜（計步不受影響）。③ 焊主板時排線不要蓋住 USB-C 與按鍵。

3 接線與原因：為什麼接這幾支腳

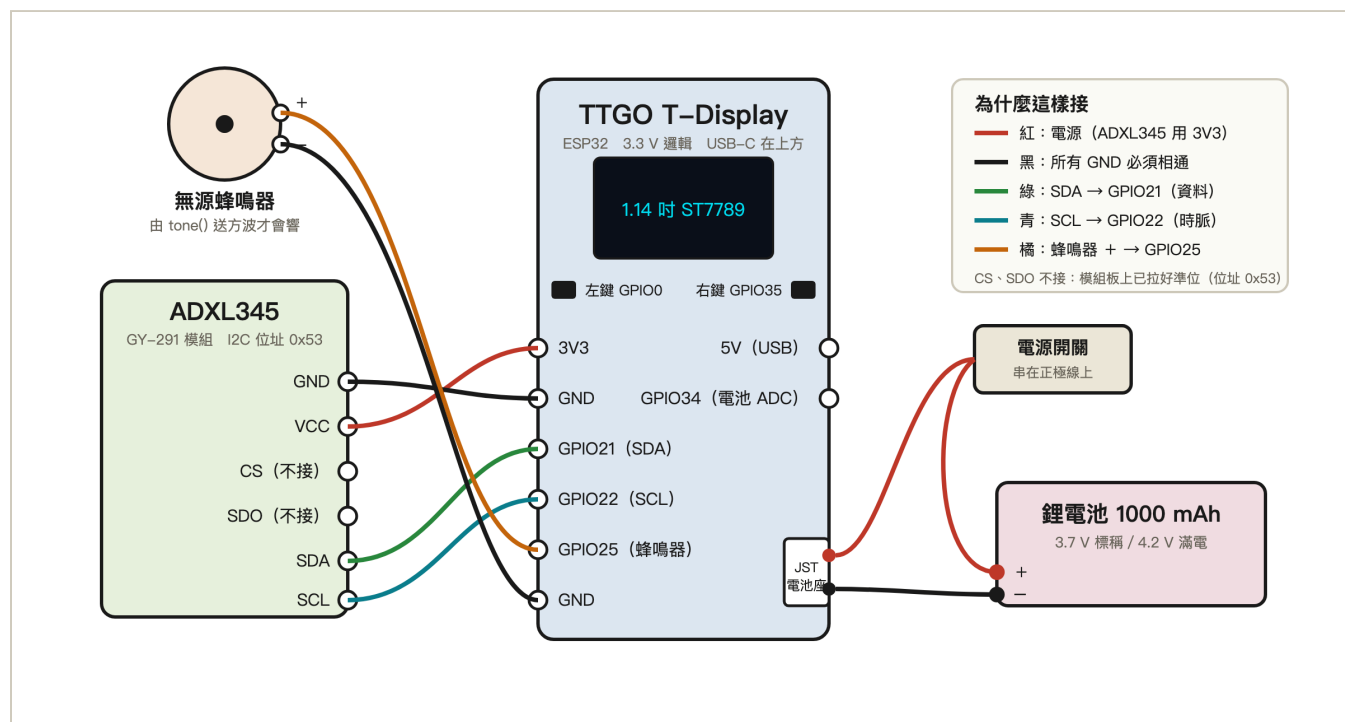


圖 3-1 完整接線。顏色對應右上角圖例；CS、SDO 兩支腳不接。

3.1 I2C 只要兩條線

是什麼

I2C 是一種「一條資料線 (SDA) + 一條時脈線 (SCL)」的匯流排，同一對線上可以掛很多顆裝置，靠位址區分。

為什麼

兩條線都是「開汲極」：裝置只會把線拉低，放手時要靠上拉電阻把線拉回高電位。GY-291 模組板上已經有上拉電阻，所以我們不用再加。

所以呢

SDA、SCL 接反就完全不通 (程式會報「找不到加速度計」)；位址對不上也不通。ADXL345 的位址由 SDO 腳決定：接地是 0x53、接高是 0x1D，模組板已把 SDO 拉低，所以不接就是 0x53，程式也是用 0x53。

3.2 為什麼 ADXL345 的電源接 3V3？

ADXL345 晶片本身只能吃 2.0~3.6 V。GY-291 模組上有一顆 3.3 V 穩壓器，所以接 5 V 也能動，但 SDA / SCL 的高電位仍是 3.3 V，直接接 3V3 最單純，也省掉穩壓器的壓差損耗。這也是為什麼在挑模組時要看「板上有沒有穩壓器」——沒有的裸晶片板接 5 V 會直接燒掉。

3.3 共地

電壓是「相對」的：ESP32 判定輸入為高電位的門檻是 $0.75 \times VDD$ （約 2.5 V）、低電位是 $0.25 \times VDD$ （約 0.8 V）——都是以板子自己的 GND 為 0 V 量的。感測器若沒和板子共地，兩邊的 0 V 會各自漂移，I2C 的高低準位就判不準、通訊直接中斷。所有零件的 GND 都要連在一起，電壓才有共同的參考點。模擬器的「接線」分頁第一條規則就是檢查這件事：蜂鳴器的負極、ADXL345 的 GND、電池負極，少接任何一條都會被標出來。

3.4 電池電壓怎麼量：GPIO34 + GPIO14 (ADC_EN)

是什麼 鋰電池最高 4.2 V，超過 ADC 能量的範圍，板上用兩顆 100 k Ω 分壓成一半接到 GPIO34，程式讀到的值要 $\times 2$ 。

為什麼 分壓電阻若永遠接著電池，會一直漏約 21 μA ($4.2 V \div 200 k\Omega$)。原廠在這條路上加了一顆 MOSFET，由 **GPIO14 (ADC_EN)** 控制：拉高才導通、才量得到。這就是我們第一版「電量亂跳」的原因。

所以呢 量測順序：GPIO14 拉高 $\rightarrow$ 等幾十微秒 $\rightarrow$ 讀 GPIO34 $\rightarrow$ GPIO14 拉低。本韌體的 `hal_battery_volts()` 就是這樣做；ESP32 用 `analogReadMilliVolts()` 讀回已校正的毫伏值，比自己乘 3.3/4095 準。

動手前先在模擬器接一次

模擬器的「接線」分頁可以拖線、按「檢查接線」，九條規則會逐條回饋（短路、GPIO 碰電源、共地、I2C 接反、位址、CS 接地切成 SPI、蜂鳴器腳位、電池經開關、極性）。接錯會真的反映到模擬：SDA / SCL 接反重開機後螢幕就出現「找不到加速度計」。

計步器模擬器

韌體 v1.0.0 (WebAssembly)

無線網路

熱點模式

名稱：計步器-GOTT

密碼：12345678

網址：http://192.168.4.1/

網頁帳號 admin，密碼預設 1234

長按 選單

左鍵 (GPIO0)

右鍵 (GPIO35)

同按 短放

同按 長按

鍵盤：← 左鍵 → 右鍵 空白鍵 同按短放 Enter 同按長按。按住滑鼠就是按著按鍵。

螢幕文字由瀏覽器以 PingFang TC 描繪；真機用的點陣字也是從同一套字型描出來的 (tools/gen_font.py)，位置與字形一致。

模擬時間 24.63 小時

速度 1x

暫停

跳到明天

重新開機

感測器

序列埠

儲存

儀表板

接線

檢查接線

清除接線

示範正確接法

點一個腳位、再點另一個腳位=接一條線；點線=刪除。接錯會反映到模擬器（找不到加速度計、蜂鳴器不響）。

全部通過。

✓ 沒有電源短路。

✓ ADXL345、蜂鳴器、電池都與開發板共地。

✓ ADXL345 VCC 接 3V3。

✓ SDA → GPIO21。

✓ SCL → GPIO22。

△ SDO 沒接：GY-291 模組板上已把 SDO 拉低，位址仍是 0x53，可以不接。

✓ 蜂鳴器 + → GPIO25、- → GND。

✓ 電池 + → 開關 → 電池座+。

✓ 電池 - → 電池座-。

模擬器狀態：加速度計 可用、蜂鳴器 可用。加速度計是開機時偵測，改完接線請按「重新開

圖 3-2 模擬器的接線關卡：示範接法檢查全部通過；SDO 不接會以「△」提醒原因，不算錯。

4 從加速度到步數：演算法怎麼想

先看效果，再講參數。下圖是把韌體裡那份真正的演算法餵進合成訊號後倒出來的中間結果：前 2 秒靜止、2~10 秒以每分鐘 110 步走路、12.5 秒時搖晃一下。

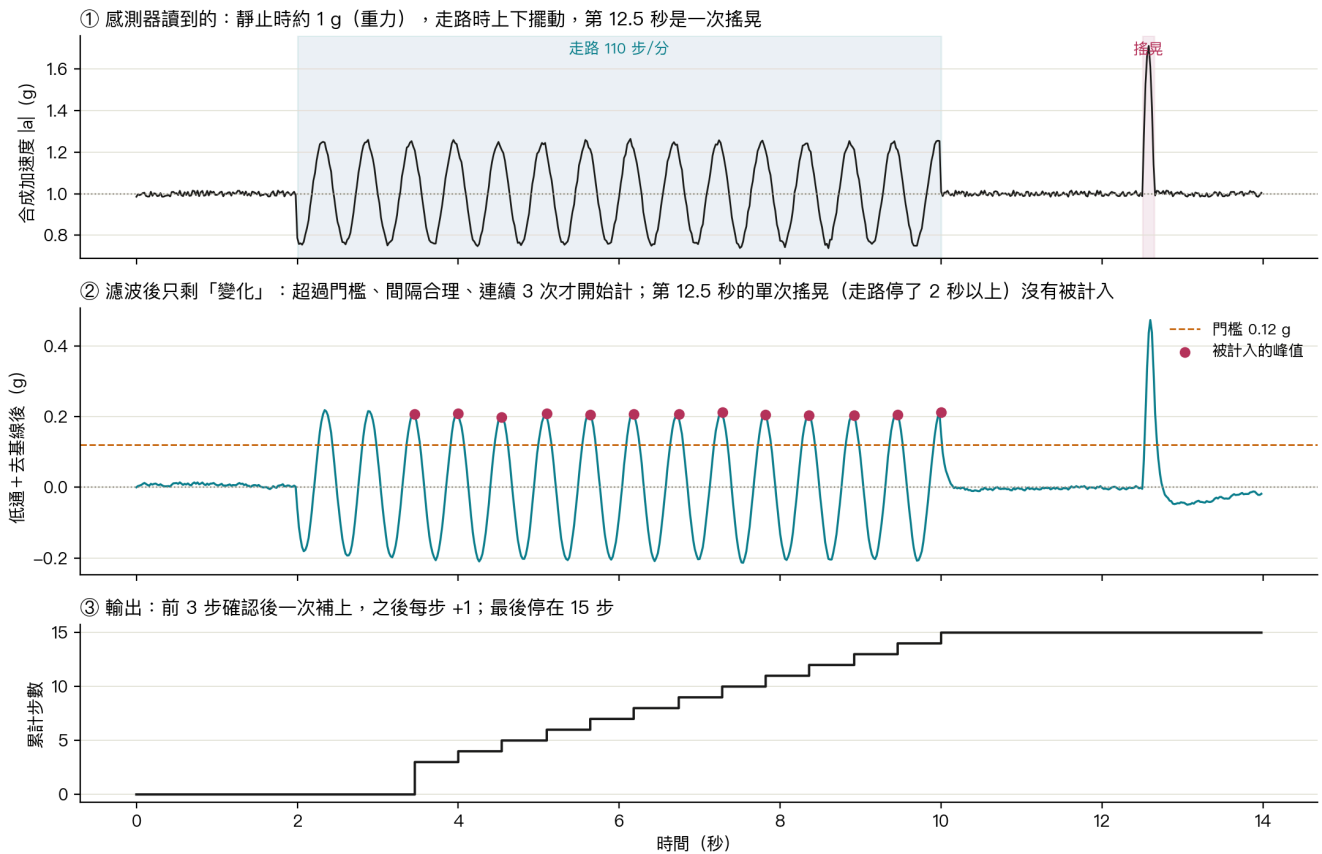


圖 4-1 上：感測器讀到的合成加速度，靜止時是 1 g。中：濾掉重力與慢變化後只剩「變化」，橘色虛線是門檻，粉紅點是被計入的峰值。下：累計步數。

4.1 三道處理

- 合成加速度** $|a| = \sqrt{ax^2 + ay^2 + az^2}$ 。用合成值而不是某一軸，裝置怎麼戴、怎麼轉都不影響。
- 低通 + 去基線** 先用低通濾掉高頻抖動（ $lp += 0.3 \cdot (|a| - lp)$ ），再減掉緩慢變化的基線（ $dc += 0.02 \cdot (lp - dc)$ ）。基線就是重力加上慢慢變化的姿勢，剩下的才是「每一步」。
數學上這兩個都是一階 IIR 低通 $y[k] = y[k-1] + \alpha \cdot (x[k] - y[k-1])$ ，截止頻率 $f_c = -\ln(1-\alpha) / (2\pi \cdot \Delta t)$ 。取樣 20 ms 時： $\alpha=0.3 \rightarrow f_c \approx 2.8 \text{ Hz}$ （走路 1.5~3 步/秒剛好留著，腳跟落地的尖銳衝擊被濾掉）； $\alpha=0.02 \rightarrow f_c \approx 0.16 \text{ Hz}$ 、時間常數約 1 秒（只跟得上姿勢的慢變化）。
- 峰值 → 間隔 → 連續確認** 訊號由升轉降且高過門檻（0.12 g）算一個候選步；和上一步間隔要在 250~2000 ms 之間（太快是抖動、太久是停下來）；連續 3 個合格候選才開始計，之後每個峰值 +1。

4.2 四個可調參數

參數	預設	調大會怎樣	調小會怎樣
峰值門檻 k_thresh	0.12 g	輕步不算、搖晃也不算	敏感，桌子震一下就計
最短間隔	250 ms	快跑 (>240 步/分) 會漏	一步內的二次震動被算成兩步
最長間隔	2000 ms	慢走較不會中斷，但停下後的一次碰撞也可能被接續計入	慢走會一直重新「連續確認」，前幾步被吃掉
連續確認	3 步	更不會誤判，起步前幾步顯示延遲	反應快，單次搖晃容易被計

一個誠實的限制

連續三次間隔 0.7 秒的搖晃，對這個演算法來說就是「走路」——它和真的走路在訊號上沒有差別。手環用的進階做法會再看頻率是否穩定、三軸能量分布，或是直接用晶片內建的計步器（第 5 節）。這正是教學上值得討論的點：演算法在「規則」與「例外」間取舍。

4.3 實驗：在模擬器裡調參數

1. 打開模擬器 → 同按短放進入「計步」→ 按「開始走路」，看步數是否在 3 秒後開始跳、每分鐘約 110 步。
2. 停止走路，按一下「搖晃一下」，等 3 秒：步數不應該變。再快速連按三下：步數會 +3——想想為什麼。
3. 切到「儀表板 → 設定」，把門檻改成 0.05，再走路與搖晃，觀察波形圖的橘色虛線與誤判。改回 0.12，再試 0.3（這時振幅 0.25 g 的走路會完全計不到）。
4. 把「振幅」滑桿拉到 0.08 g（像是把裝置放在口袋深處輕走），看哪個門檻才計得到；討論「敏感」與「誤判」的取舍。
5. 燒進真機後，用手機連熱點開儀表板，實際走 100 步比對誤差，再回頭調參數。

4.4 距離與卡路里是怎麼算的

距離 = 步數 × 步幅；活動消耗 $\approx$ 距離 (km) × 體重 (kg) × 1.036。這是常見的步行概估公式，誤差不小——步幅會隨速度改變、坡度沒有被考慮。把這兩個數字看成「參考」而不是「量測」，是使用任何計步器都該有的素養。

新版把「個人資料」做進設定選單與網頁：性別、年齡、身高、體重。步幅預設由身高推算（男 $0.415 \times$ 身高、女 $0.413 \times$ 身高，常見的經驗係數），也可手動輸入；基礎代謝用 Mifflin—St Jeor 算出

來顯示在裝置與網頁；**今日總消耗估計** = 活動消耗 + 基礎代謝 × 今日已過的時間比例。這三個數字分開顯示，學生才看得出「手環的數字為什麼比較大」是因為加了基礎代謝。

為什麼手環顯示的卡路里比我們多很多？

本裝置算的是**活動消耗**（走路額外燒掉的）。市售手環顯示的「總消耗」還加上**基礎代謝率**

（BMR，躺著不動也在燒的能量），例如 Mifflin—St Jeor 公式：男 $BMR \approx 10 \times \text{體重} + 6.25 \times \text{身高} - 5 \times \text{年齡} + 5$ （女 -161 ）kcal/天，60 kg、170 cm、16 歲男生約 1,620 kcal/天，平均每小時 67 kcal——光這一項就比走 1 小時的活動消耗還多。不先分清楚這條邊界，學生會以為自己的演算法少算了。

4.5 水平儀：同一顆感測器的另一種用法

靜止時三軸量到的是重力向量，向量的方向就是「哪邊朝下」。俯仰角 $\text{pitch} = \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$ 、橫滾角 $\text{roll} = \text{atan2}(a_y, a_z)$ ，再做一點平滑。這說明了為什麼第 5 節選型時「只要加速度計」就夠：傾斜量測不需要陀螺儀——除非你要量「轉得多快」或在移動中量姿態。



裝置綁在腳踝、手腕或放口袋，訊號都不同：腳踝震動最強、手腕疊加手臂擺動、口袋最接近身體質心。實測時請固定一種戴法再比較參數。（AI 生成示意圖）

5 模組比較與選型：換一顆行不行

加速度計怎麼選：先問四個問題，再看型號

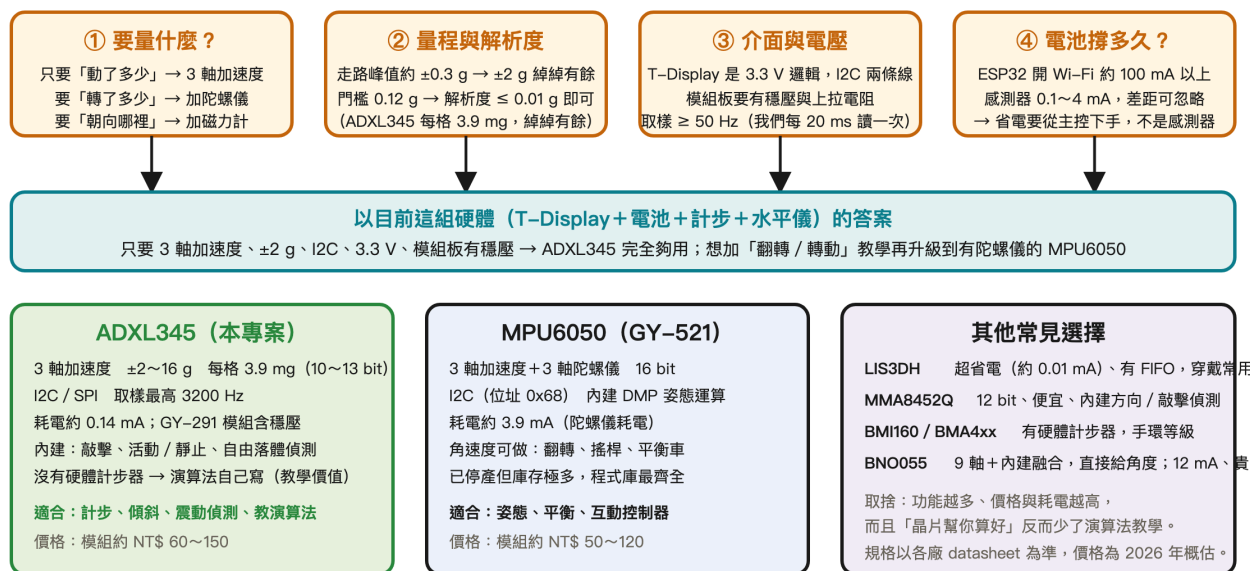


圖 5-1 選加速度計先問四個問題。對目前這組硬體而言，ADXL345 已經足夠；要教「轉動」再上 MPU6050。

5.1 加速度計 / 慣性感測器

型號	量什麼	介面 / 位址	解析度	耗電 (量測中)	內建功能	對本專案的評價
ADXL345	3 軸加速度	I2C $0x53$ / $0x1D$ 、SPI	3.9 mg / 格	約 0.14 mA	敲擊、活動 / 靜止、自由落體	夠用；演算法自己寫，教學價值最高
MPU6050	加速度 + 陀螺儀	I2C $0x68$ / $0x69$	16 bit	約 3.9 mA	DMP 姿態融合	多了轉動；已停產但庫存多、資料最齊
LIS3DH	3 軸加速度	I2C $0x18$ / $0x19$ 、SPI	12 bit	$0.002\sim 0.011\text{ mA}$	FIFO、點擊偵測	最省電；想做「電池撐一週」的版本再換
MMA8452Q	3 軸加速度	I2C $0x1C$ / $0x1D$	12 bit	$0.006\sim 0.165\text{ mA}$	方向、敲擊、自由落體	便宜替代；注意位址可能與 ADXL345 的 $0x1D$ 撞
BMI160 / BMA456	加速度 (+ 陀螺儀)	I2C / SPI	16 bit	約 0.18 mA	硬體計步器	手環等級；晶片直接給步數，少了演算法教學

BMI270	加速度 + 陀螺儀	I2C 0x68 / 0x69、 SPI	16 bit	約 0.69 mA（雙感 測）	硬體計步器、手 勢、姿態辨識	現代穿戴主流（Bosch）； 省電，但步數由晶片給， 少了演算法教學
ICM-42688-P	加速度 + 陀螺儀	I2C、SPI	16 bit	約 0.88 mA	低雜訊（加速度 70 $\mu\text{g}/\sqrt{\text{Hz}}$ ）、2 KB FIFO	無人機、工業級；對計步 是殺雞用牛刀
BNO055	9 軸 + 融合	I2C 0x28	—	約 12 mA	直接輸出角度、 四元數	貴、耗電；適合機器人姿 態，不適合計步器

規格取自各廠資料手冊的典型值，不同模組板（有無穩壓、上拉）會影響實際接法與耗電；價格（台灣零售，2026 年）：ADXL345 模組約 NT\$ 60~150、GY-521 約 NT\$ 50~120、LIS3DH 模組約 NT\$ 100~250、BNO055 模組約 NT\$ 600~1200。

5.2 選擇條件（針對目前這組硬體）

條件	這組硬體的需求	判斷
量測種類	計步只要「動了多少」；水平儀只要重力方向	3 軸加速度即可，陀螺儀是加分不是必要
量程	走路峰值約 $\pm 0.3\text{ g}$ 、跑步約 $\pm 1\text{ g}$	$\pm 2\text{ g}$ 檔位足夠；留 $\pm 4\text{ g}$ 給跑步
解析度	門檻 0.12 g，要能分辨 0.01 g 等級的變化	3.9 mg / 格綽綽有餘
取樣率	程式每 20 ms 讀一次（50 Hz）	所有候選都遠超過；ADXL345 設 100 Hz 頻寬
介面與電壓	T-Display 是 3.3 V 邏輯，I2C 預設 21 / 22	模組板要有穩壓與上拉；避免位址互撞
耗電	ESP32 本身 80~240 mA	感測器 0.1~4 mA 都可忽略；省電要靠關 Wi-Fi、降頻、睡眠
程式庫與資料	學生要能查得到範例	ADXL345、MPU6050 資料最多；本專案直接讀暫存器，不依賴外部函式庫
教學目標	要讓學生「看到演算法」	選沒有硬體計步器的型號，才有東西可教

5.3 顯示器

方案	規格	優點	代價	適合
T-Display 板載 ST7789	1.14 吋、135×240、 彩色、SPI	不用接線、彩色、速度快	佔掉 6 支 GPIO；中文字型要自己想辦法（第 7 節）	本專案

SSD1306 OLED	0.96 吋、128×64、單色、I2C	兩條線、省電、便宜	小、單色；中文一行只放 8 個字	只顯示數字的簡化版
無螢幕	只靠蜂鳴器與網頁	最省電、最便宜	沒有手機就看不到	純感測記錄器
電子紙 2.13 吋	250×122、SPI	陽光下可讀、不耗電	更新慢（秒級）、不能動畫	一天看幾次的步數牌

5.4 主控板

板子	核心	無線	電池	程式語言	對本專案
TTGO T-Display	ESP32 雙核 240 MHz、4 MB	Wi-Fi、藍牙	有充電與電池座	Arduino C++ / MicroPython	一片板子解決螢幕、按鍵、充電、網頁
Raspberry Pi Pico W	RP2040 雙核 133 MHz、2 MB	Wi-Fi	無充電電路	MicroPython / C++	要另接螢幕與充電模組；MicroPython 上手快
ESP32-C3 SuperMini	單核 160 MHz、4 MB	Wi-Fi、BLE	無	Arduino / MicroPython	最小最便宜；要另接螢幕
Arduino Nano (AVR)	8 位元 16 MHz、32 KB	無	無	Arduino C++	放不下中文網頁與 Wi-Fi，不適合

5.5 電源

是什麼 1000 mAh 鋰電池。以 Wi-Fi 開著平均 120 mA 估，約 8 小時；若改成「只有開網頁時才開 Wi-Fi」可以撐一整天以上。

為什麼 耗電大戶是 ESP32 的無線電，不是感測器也不是螢幕背光（背光約 20 mA）。所以省電的第一步是管理 Wi-Fi，而不是換更省電的加速度計。

所以呢 鋰電池不要放電到 3.3 V 以下、不要刺穿或壓到、充電時不要離開；程式裡電量 0% 設在 3.3 V 就是這個原因。開關串在正極線上，隨時能徹底斷電。

給這組硬體的結論

加速度計維持 ADXL345；蜂鳴器維持無源（才能放旋律）；螢幕用板載的；電池 1000 mAh 夠一天教學使用。真正值得升級的是軟體：加「閒置 5 分鐘自動關 Wi-Fi」能讓續航翻倍；想教轉動與姿態時，再把感測器換成 MPU6050——接線完全一樣（I2C 21 / 22），只改讀暫存器那一小段程式。

6 軟體架構與模擬器：先在電腦上跑

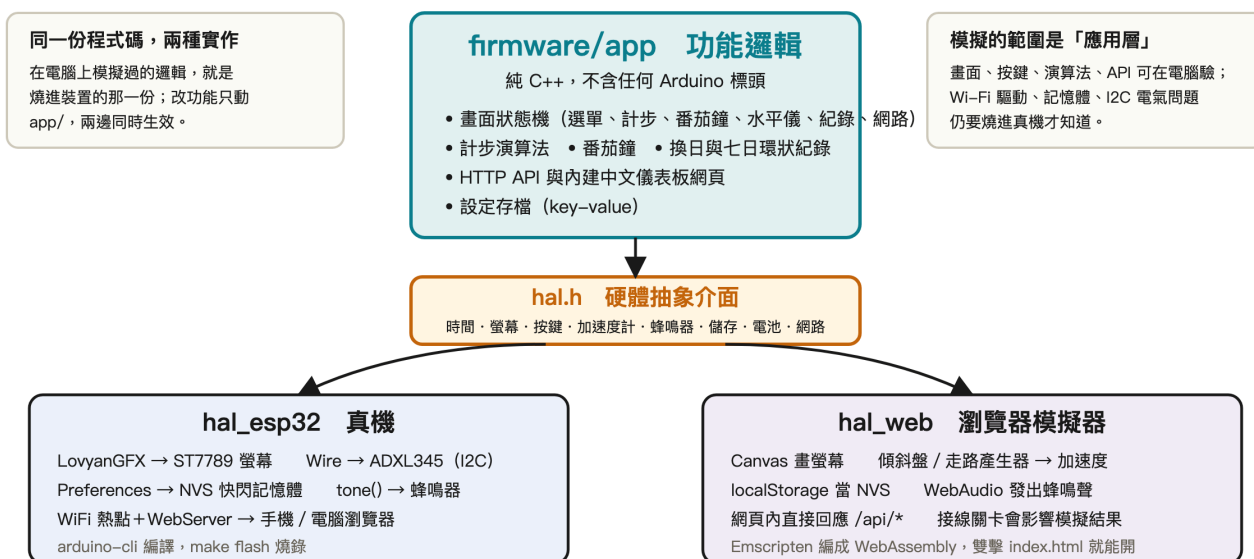


圖 6-1 功能邏輯只透過 hal.h 碰硬體，所以同一份程式既能燒進裝置，也能編成 WebAssembly 在瀏覽器執行。

6.1 為什麼要分層

是什麼

firmware/app/ 是功能（畫面、計步、番茄鐘、API），hal.h 是一張「硬體能做什麼」的清單，hal_esp32/ 與 sim/hal_web/ 是這張清單的兩種實作。

為什麼

學生最常卡在「改一行要等兩分鐘燒錄、再盯著小螢幕看」。把功能和硬體分開，功能就能在電腦上以 600 倍速測，番茄鐘 25 分鐘只要 3 秒。

所以呢

模擬器通過不代表真機一定通過——Wi-Fi 驅動、記憶體不足、I2C 接觸不良都不在模擬範圍。模擬器負責「邏輯對不對」，真機負責「硬體行不行」。

6.2 模擬器介面



圖 6-2 左：裝置螢幕、兩顆按鍵、同按輔助鈕、模擬時間與速度。右：感測器（傾斜盤、走路產生器、波形）、序列埠、儲存（NVS）、儀表板、接線五個分頁。

分頁	可以做什麼	對應真機的什麼
感測器	拖圓盤改傾斜角；開始走路（步頻、振幅可調）；搖晃一下；看去重力後的波形與門檻線；電池電壓滑桿	ADXL345 的三軸讀值、GPIO34 電池電壓
序列埠	看程式印出的訊息（開機、存檔、換日、網頁操作）	USB 序列埠 115200 baud
儲存	看每個 key 的原始位元組與解讀；清除＝模擬抹除	Preferences / NVS 分區
儀表板	就是裝置內建的網頁，直接對模擬器的程式發 API	手機連熱點後看到的頁面
接線	拖線、檢查、示範正確接法；結果影響模擬	實際接線前的演練

6.3 建議的工作流程

1. 接線演練：在「接線」分頁接一次、檢查、重開機，確認加速度計被偵測到。

2. 功能走一遍：五個畫面都進去看；番茄鐘用 600x 看完整一輪；「跳到明天」看七日紀錄換日。
3. 改程式：只動 `firmware/app/`。 `make sim-build` 幾秒後重新整理頁面。
4. 自動驗收：`node tools/qc_sim.mjs`，16 項機器判定（走路計步 40~70 步、單次搖晃不計、60 秒自動存檔、儀表板一致、番茄鐘到時響鈴、換日、接線規則.....）；
`build/test_pedometer` 跑演算法的 9 組情境。
5. 燒錄：`make build`（會先自動產生字型）→ `make flash PORT=...`。燒錄腳本只接受允許清單裡的埠。
6. 真機驗證：序列埠看開機訊息；手機連熱點、開儀表板同步時間；實走 100 步比對。

為什麼要有「機器可判定」的驗收

「看起來正常」會漏掉很多事。這次 16 項驗收就抓到三個真正的錯誤：模擬迴圈被瀏覽器降速（30 秒只計到 4 步）、同按手勢用真實時間計時（加速時被判成長按）、重開機後時間倒退被當成換日。每一個都不是看畫面能發現的。

6.4 時間從哪裡來

ESP32 沒有電池時鐘，重開機就不知道今天幾號。新版的做法：手機或電腦一打開儀表板，網頁就把目前時間送給裝置（`POST /api/time`）；裝置記住「同步當下的開機毫秒數」，之後用差值推算。七日紀錄的「換日」只在推算日期往後跳時觸發——重開機後時間會暫時倒退，不能誤判成換日（這也是驗收抓到的錯之一）。螢幕右上角顯示「時間未同步」就是在提醒使用者開一次網頁。

7 失敗路徑：我們踩過的坑

失敗不是附錄，是教材。下面每一條都是「做了什麼 → 哪裡不對 → 改成什麼 → 學到什麼」。

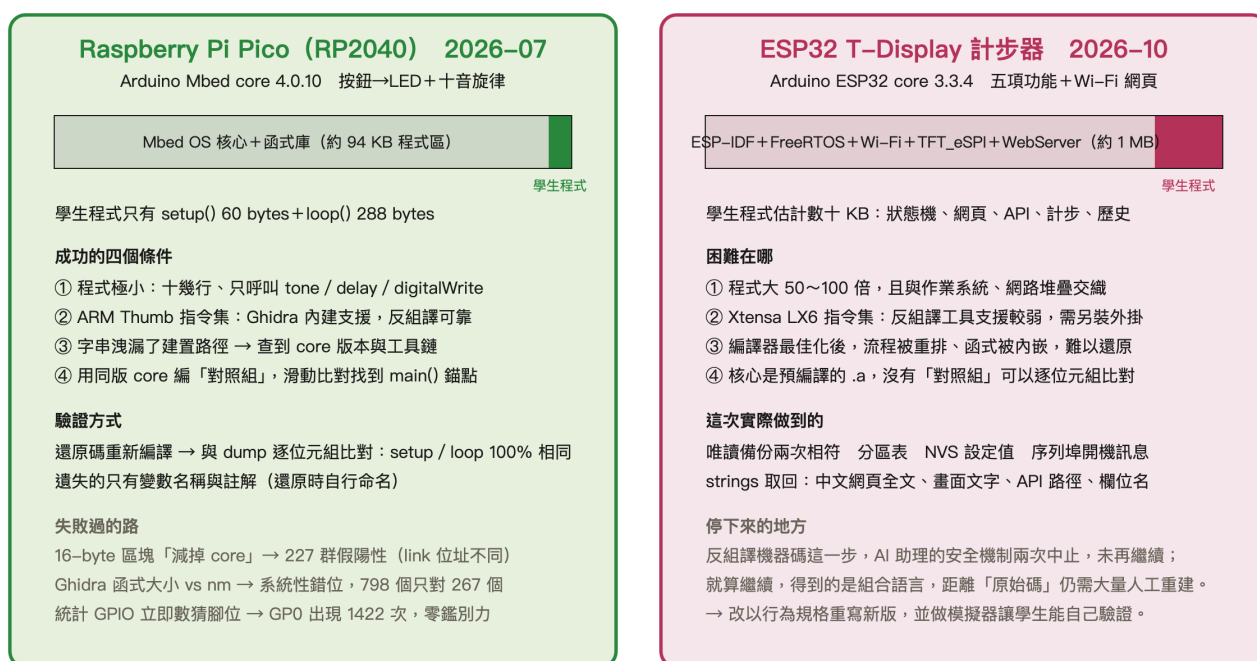
嘗試	失敗	改用	學到
用 LovyanGFX 內建的繁中字型（12、16、24 三個字級）	韌體 3.18 MB，程式分區只有 1.28 MB，連不上	建置時掃描程式用到的 271 個字，從系統字型描成點陣，只要 35 KB	「全字集」是為通用而設計；專案只需要自己用到的字。自動化（每次 make 重掃）才不會漏字
用 460800 baud 讀整片快閃記憶體	讀到 15% 斷線「Serial data stream stopped」	降到 115200，4 MB 約 6.5 分鐘，兩次讀取 SHA256 一致	速度與可靠度的取捨；備份一定要讀兩次比對
直接用現成的線上模擬器（Wokwi）	沒有 T-Display、沒有 ADXL345，裝置網頁要付費方案才連得到	自己做：同一份 C++ 編成 WebAssembly	工具不合需求時，評估「自己做」的成本；分層架構讓這件事變得可行
模擬迴圈用 requestAnimationFrame 驅動	瀏覽器在背景或無頭模式會把它壓到極低，30 秒走路只計到 4 步	改用 setInterval 驅動，rAF 只負責出畫面	先用原生單元測試證明演算法正確（55 步），才確定問題在驅動層——隔離問題
「同按短放」輔助鈕用真實時間 150 ms 放開	10× 加速時裝置端看到按了 1.5 秒，被判成長按回選單	按鍵持續時間改用模擬時間計算	時間有兩種：真實的與模擬的。所有「多久」都要問是哪一種
搖晃三下、間隔 0.7 秒，預期不計步	被計了 3 步	把測試改成單次搖晃；三下 0.7 秒本來就是走路步頻	測試寫錯和程式寫錯一樣常見；先想清楚「物理上該發生什麼」
重開機後沿用上次同步的時間	開機毫秒數歸零 → 推算日期倒退一天 → 觸發「換日」把今天清掉	只有日期往後跳才換日	邊界條件：時間不一定單調遞增

測深度睡眠時另開一個序 列埠程式監聽喚醒	一開埠板子就重 開機（rst:0x1 POWERON）， 測試結果看起來 像「RTC 時間沒 接回」	單一連線從同步、關機到喚醒跑 到底 （ <code>tools/sleep_test.py</code> ）	USB 轉序列晶片開埠會抖 動 DTR / RTS，而那正是自 動燒錄用的重置線；量測工 具本身也會干擾被量的東西
直接讀 GPIO34 當電池 電壓	讀值不對：T- Display 的分壓 電路要由 GPIO14 （ADC_EN）啟 用	查官方腳位圖，開機時把 GPIO14 拉高	板子的「特殊腳」要看原廠 資料，不能只憑 Arduino 預 設腳位想像
原作韌體反組譯	見第 8 節	以行為規格重寫	見第 8 節

8 兩個逆向案例：Pico 成功、ESP32 停下

這學期我們有兩次「學生電腦壞了、原始碼不見了，只剩板子」的經驗。一次在 Raspberry Pi Pico 上完整還原了程式，這次在 ESP32 上沒有。差別不在運氣。

同樣是「從晶片抓回程式」，為什麼一個成功、一個停下來？



共同教訓：編譯後的機器碼不會保留原始碼。原始碼要放進版本控制（例如 GitHub），不要只存在一台電腦裡。

圖 8-1 兩個案例的對照。注意兩次都是 Arduino C++ 編譯後的機器碼，不是直譯式語言；Pico 那次成功是因為條件剛好全部成立。

8.1 先更正一個常見印象

很多人記得 Pico 那次是「抓回 Python 程式」。查了當時的紀錄（[pico-recover](#)），那片 Pico 上跑的是 **Arduino Mbed RP2040 核心**編譯的 C++，不是 MicroPython。還原是靠 `picotool save` 倒出 2 MB 快閃記憶體、Ghidra 反組譯、再用同版核心編譯對照組逐位元組比對，最後 `setup()` 與 `loop()` 100% 相符。

如果當時真的是 MicroPython，事情會更簡單：MicroPython 把 `main.py` 等原始碼以明文存在板上的檔案系統，用 Thonny 或 `mpremote fs cp` 直接複製出來就好，連反組譯都不用。這是直譯式語言在「可回收性」上的本質優勢——代價是執行速度與記憶體。

8.2 為什麼 Pico 可以

條件	Pico 那次	ESP32 這次
學生程式有多大	setup 60 bytes + loop 288 bytes，十幾行	估計數十 KB：狀態機、內嵌網頁、API、計步、歷史
和系統的糾纏程度	只呼叫 tone / delay / digitalWrite，與核心界線清楚	FreeRTOS 多工、Wi-Fi 事件回呼、WebServer 處理函式散在各處
指令集與工具	ARM Thumb-2，Ghidra 內建支援成熟	Xtensa LX6，需另裝外掛，支援較弱
有沒有「對照組」	字串洩漏建置路徑 → 查到核心版本 → 同版編譯空白 / blink 程式，滑動比對找到 main() 錨點	核心是預編譯的 .a (arduino-lib-builder)，版本查得到，但 1 MB 中要定位數十 KB 的使用者程式沒有可靠錨點
驗證方式	還原碼重新編譯 → 與映像逐位元組比對	無法做到同等強度的驗證
結果	原始碼 100% 等價（只失去變數名與註解）	取回明文的網頁與字串；機器碼反組譯這一步未進行

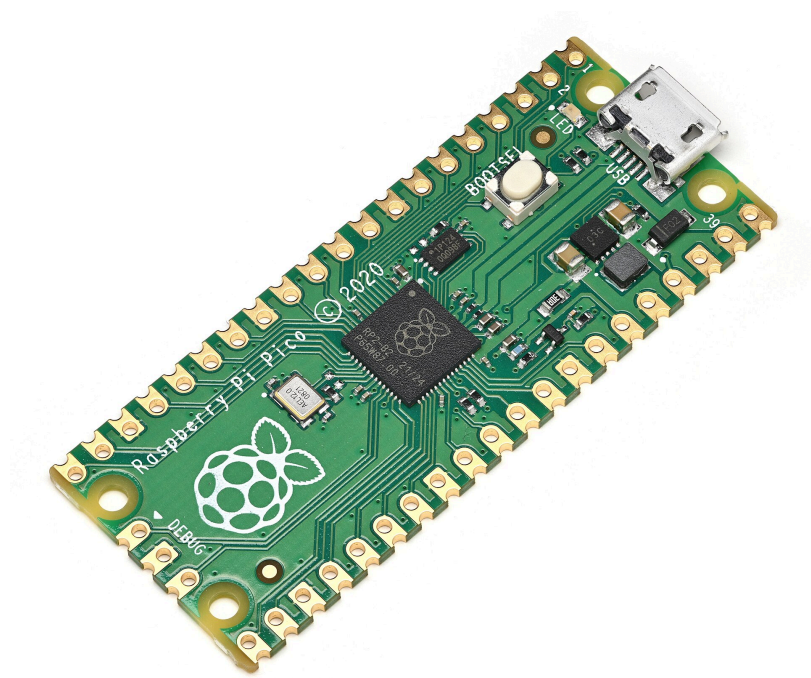
8.3 這次真正發生的事

唯讀備份、分區表、NVS 裡存的設定（步數 164、工作 6 分 / 休息 3 分）、序列埠開機訊息、以及用 `strings` 取回的整頁中文網頁與全部畫面文字，這些都拿到了。到了「把 Xtensa 機器碼反組譯回邏輯」這一步，協助分析的 AI 工具基於安全政策兩次中止，之後沒有再嘗試。即使繼續，得到的會是組合語言與偽碼，要還原成可讀的 C++ 仍需大量人工，而且無法像 Pico 那樣用「重新編譯逐位元組比對」來證明還原正確。

所以我們改走另一條路：把取回的畫面文字、API、資料欄位當成**行為規格**，重寫一份中文版，並做模擬器讓學生自己驗證行為。新版不是原作的複製品，而是以原作為規格的重新設計。

給學生的三個教訓

① 編譯後的機器碼不保留原始碼——變數名、註解、結構在編譯時就消失了。② 原始碼要放進版本控制（GitHub、GitLab）或至少雲端同步，「只存在一台電腦」等於沒有備份。③ 對不熟悉的板子，動手前先做**唯讀**備份並讀兩次比對；拖一個檔案進去就可能覆寫原程式（Pico 的 RPI-RP2 磁碟就是這樣）。



Raspberry Pi Pico (RP2040)。按住 BOOTSEL 插上電腦會變成一個磁碟，`picotool save -a` 可以唯讀倒出整片快閃記憶體。

9 延伸專案

六個由淺到深的延伸，完整程式與驗證方式都在模擬器網站的「延伸專案」頁 (sim/web/extensions.html)，可直接下載。

#	專案	做什麼、為什麼值得做	已做的驗證
A	雙感測器比對與校正 進階	ADXL345 (0x53) 與 MPU6050 (0x68) 掛同一條 I2C，貼在一起同步讀；兩顆各跑同一份計步演算法；按左鍵做六面校正（每軸 ± 1 g 求偏移與刻度）。電腦端腳本算靜止偏移、雜訊、波形相關係數、步數一致性並畫圖。回答「換一顆差多少」這個問題用數據，不用感覺。	arduino-cli 編譯通過；分析腳本可執行
B	久坐提醒 入門	原作說明書寫了但沒做：連續 60 分鐘沒步數就響鈴，之後每 15 分鐘再提醒，番茄鐘進行中暫停。純 C++ 模組，三行接進 app.cpp；模擬器 600× 一分鐘就能看完一小時。	單元測試 9/9
C	步頻穩定度 挑戰	多要求最近三個步伐間隔的變異係數 $\leq 25\%$ 才開始計。忽快忽慢的五下搖晃：原版計 5 步、穩定版 0 步；等距的三下搖晃兩版都擋不掉——因為它在物理上就是走路步頻。這個「擋不掉」本身就是討論題。	對照測試 6/6
D	省電兩招 進階	閒置 5 分鐘自動關 Wi-Fi、按鍵重開；長按左鍵 3 秒進深度睡眠（ESP32 約 10 μ A）、右鍵喚醒。這是 2.1 節第三種關機方式的實作，兼顧省電與充電。有電流表就量三種狀態。	arduino-cli 編譯通過
E	電腦端後台 進階	原作說明書的架構：裝置連家用 Wi-Fi、每分鐘 POST 步數到電腦，電腦存 CSV 並顯示全班總表。只用 Python 內建模組。與新版「裝置當伺服器」對照：幾個人用哪種、幾十台用哪種。	Python 可執行；裝置端編譯通過
G	遊戲卡帶架構 入門～挑戰	主選單多一項「遊戲」，每個小遊戲是一張卡帶（名稱 / begin / update / draw），計步在背景照跑。已內建擲筊、骰子、量角器三張；新卡帶只要繼承類別加一筆，模擬器直接驗。第 10 節詳述。	主韌體內建；模擬器驗收 5 項

F	換成 MPU6050 進階	主韌體已內建：編譯時加 <code>-DACCEL_MPU6050</code> (<code>make build-mpu</code>) 就改讀 MPU6050。接線完全相同，只有讀暫存器那一小段不同——這就是硬體抽象層的价值。做完 A 再做 F 最順。	<code>make build-mpu</code> 編譯通過
---	------------------	--	-------------------------------------

討論題

1. 如果加速度計只給你 X 軸，計步會發生什麼事？裝置轉 90 度呢？
2. 手環的計步器幾乎都用「晶片內建計步」。對產品是好事，對學習呢？
3. 裝置沒有時鐘，我們靠網頁同步。還有哪些方法？各要付出什麼（成本、耗電、接線）？
4. 專案 C 擋不掉等距搖晃。還有什麼特徵能分辨「走路」和「有節奏地搖」？（提示：三軸能量分布、重力方向是否穩定）
5. Pico 的還原成功了，但花了一整天。如果當初多做哪一件事，這一天可以省下來？

專案 A 的做法（給要帶學生做的老師）

- ① 兩顆模組用雙面膠貼在同一塊小板上、軸向對齊，四線並接到 3V3 / GND / 21 / 22，MPU6050 的 AD0 不接。
- ② `make ext NAME=dual_sensor_compare` 編譯、`make flash-ext` 燒錄。
- ③ `python3 tools/compare_sensors.py /dev/cu.usbserial-XXXX --secs 60`：先靜置 10 秒、走 30 秒、再靜置；報告會列出兩顆的 $|a|$ 平均（應為 1.000 g）、雜訊、相關係數、步數差。
- ④ 按左鍵進六面校正，再收一次，比較校正前後偏移是否收斂到 0.01 g 內。

10 遊戲化方向：讓學生想戴著走

這一節整理自另一場 AI 討論（Mac Studio 上的 Claude），參考專案連結已逐一查證存在；「卡帶架構」與第一批卡帶已實作進主韌體。

10.1 為什麼要遊戲化

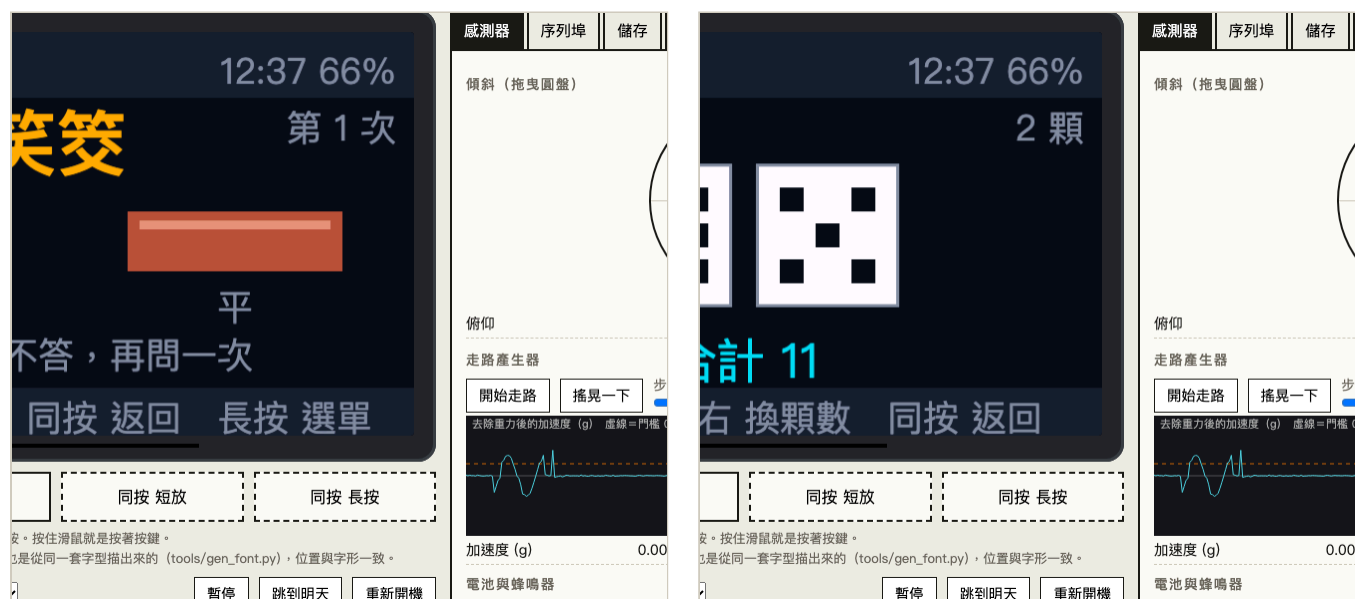
計步器的目的就是「讓人多走」。只顯示數字很快就沒人看了；把步數變成遊戲資源、把走動變成遇見同學，動機才會留下來。而且遊戲是學生最有感的程式題——狀態機、亂數、計時、無線通訊都能順著做。

10.2 架構：主選單多一項「遊戲」，每個遊戲是一張「卡帶」

是什麼 `games.h` 定義 `Cartridge`：名稱、提示、`begin`、`update(GameCtx)`、`draw`。框架每圈把加速度、傾角、左右鍵事件、今日步數包成 `GameCtx` 餵給前景卡帶；兩鍵同按短放回卡帶列表、長按回主選單由框架處理。

為什麼 計步本來就在主迴圈背景跑，遊戲只是多一個前景畫面，所以玩遊戲時步數照算（模擬器驗收有這一項）。卡帶不碰硬體，所以寫完直接在模擬器驗，不用燒。

所以呢 新增一個遊戲 = 新增一個類別、在 `CARTRIDGES` 陣列加一筆。第一批做了三張：擲筊（甩一下→聖 / 笑 / 陰筊，各配旋律）、骰子（1~3 顆）、量角器（傾角直讀、可歸零）。



擲筊：甩一下裝置，兩塊筊各自隨機平面或凸面朝上；一平一凸是聖筊（50%）、兩平是笑筊、兩凸是陰筊。亂數用加速度雜訊攪拌。

骰子：左右鍵換 1~3 顆，甩一下擲出。甩動偵測是「 $|a|$ 偏離 1 g 超過 0.6 g、靜止 350 ms 後才出結果」，避免一甩出好幾次。

順手拆解一個商品：Game & Watch: Super Mario Bros. (2020) 的時鐘

它的時鐘畫面是「上面數字、下面一條關卡」：主角一直往右走、出畫面再從左邊進來，右邊會掉敵人，每整點有角色跳起來撞數字，特定時刻（例如 05:55）數字變金幣，一天藏了 35 個彩蛋，按鍵能換三種背景。本版待機畫面借用這個**結構**——時刻在上、場景在下、以「時間事件」當彩蛋——但把彩蛋改成計步器的語言：**走路才會加速捲動、每 100 步噴金幣**。角色與場景都自己畫（12×16 像素圖），沒有使用任天堂的圖像。這也是一個好作業：挑一個喜歡的商品，拆出它的結構，再換成自己的內容。

10.3 A 路線：保留計步器，加遊戲性

玩法	怎麼做	用到的教學點 / 注意
賽博寵物	走路就是餵食與經驗值，累計步數達標進化；久坐一小時寵物變虛弱（接延伸專案 B）。參考 ESP32-TamaPetchi、Piko。	狀態機、NVS 存檔、像素動畫。 c.steps 已在 GameCtx 裡。
擦身而過 (StreetPass)	每台用 ESP-NOW 定時廣播自己的名片，別台經過時自動收下，集滿全班圖鑑；老師、別班算稀有卡。不需要校園 Wi-Fi。	無線通訊。 注意 ：ESP-NOW 與 Wi-Fi 熱點 / 連線必須在 同一個頻道 (Espressif 文件)；要每隔幾秒才廣播一次，否則耗電。需在 hal.h 加廣播 / 接收介面。
步數探索 RPG	步數當體力，每走幾百步觸發一次遭遇，敲兩下裝置「捕捉」，收集品存 Flash。參考 picowalker。	ADXL345 內建敲擊偵測 (TAP 暫存器)；亂數與機率。
全班打 Boss	延伸專案 E 的後台已有排行榜，再加一隻全班共用的 Boss，血量由全班總步數扣。	合作而非只比較；後端彙總。
成就徽章	連續 7 天達標、單日一萬步、凌晨還在走……	七日紀錄已有資料；只差判定與畫面。

10.4 B 路線：同一套硬體改成別的裝置

裝置	用到什麼	適合的教學點
擲筊 / 骰子 ✓ 已做	甩動偵測 + 亂數	第一堂課就能完成；有在地文化梗
量角器 / 水平儀 ✓ 已做	傾斜角換算	工具類，很快做完

傾斜迷宮、傾斜打磚塊	三軸加速度→傾斜角→球的速度	X / Y / Z 軸與三角函數；參考 VolosR 的 T-Display 遊戲改成傾斜操控
西部決鬥（雙人）	ESP-NOW + 揮動偵測：嗶聲後先揮的人贏	無線通訊與時間同步
節奏遊戲	晶片內建敲擊偵測 + 蜂鳴器節拍	中斷與計時
跑步配速器	蜂鳴器依目標步頻打拍，對照實際步頻	體育課可直接用（接體育班課程）
藍牙簡報翻頁器	模擬 BLE 鍵盤，傾斜或敲擊翻頁	老師自己用得到；BLE HID
跌倒、防盜警報	晶片內建自由落體與動靜偵測	長照或社會議題的設計題

為什麼不用藍牙來「先連線再帶去網頁」

想法很直覺：手機先用藍牙找到裝置，再自動加入它的 Wi-Fi 並開網頁。但三個地方卡住：① 手機作業系統不允許周邊裝置替手機加入網路（只有原生 App 能做，而且仍會跳確認）；② 瀏覽器用藍牙（Web Bluetooth）只有 Android Chrome 支援，iPhone 不行；③ ESP32 的藍牙堆疊要多 300~400 KB，目前韌體已占 89%，要換分區。所以本版改用「螢幕 QR + 強制門戶」：不用 App、兩種手機都通、不增加 Flash。藍牙真正值得做的是「配網」——用 App 把家用 Wi-Fi 帳密傳給裝置（Espressif 有現成的 BLE Provisioning），讓裝置連上區網、用 mDNS 名稱開網頁，這留給延伸專案。

10.5 建議的開發順序與可借用的開源專案

1. 擲筊 / 骰子（已做）：一堂課，先有成就感、也把卡帶架構學起來。
2. 賽博寵物 + 擦身而過：最能留住學生——寵物靠步數長大，也靠「認識朋友」解鎖造型。
3. 傾斜迷宮或西部決鬥：給進度快的組別當挑戰。

類型	專案（已查證 2026-10）	可借用的部分
T-Display 遊戲	VolosR SpaceWars （T-Display 太空射擊）、 GameTTG0s3 （S3 版）； github.com/VolosR	直接跑得起來；把按鍵操控改成傾斜就接上感測器
電子雞	CyberXcyborg ESP32-TamaPetchi （有 LIS3DH 傾斜小遊戲）； hackaday.io TamaFi	寵物狀態機、成長與進化設計
計步 + 寵物	Iloke-Alusala ESP_Piko_Firmware （MIT；LIS331 計步 + 動作辨識動畫）； mamba2410 picowalker （RP2350 Pokéwalker 重製）	「步數換遊戲資源」、動作狀態（走 / 慢跑 / 衝刺）的門檻法

傾斜遊戲	sakshams7903-eng The-Endless-Maze (XIAO ESP32-S3+OLED，非 T-Display)	傾斜控制、隨機產生迷宮
------	---	-------------

借用開源程式要看授權（上列多為 MIT / GPL），在教材與程式註解標明出處。

附錄

A 腳位總表

訊號	T-Display	對接
I2C SDA	GPIO21	ADXL345 SDA
I2C SCL	GPIO22	ADXL345 SCL
3.3 V	3V3	ADXL345 VCC
GND	GND	ADXL345 GND、蜂鳴器 -
蜂鳴器	GPIO25	蜂鳴器 +
左鍵 / 右鍵	GPIO0 / GPIO35	板載
電池電壓	GPIO34 (×2)	板載分壓，GPIO14 拉高才啟用
電池	JST 電池座	+ 經開關、- 直接

B HTTP API (帳號 admin，密碼預設 1234)

路徑	方法	內容
/	GET	中文儀表板
/api/status	GET	steps、calories、distance_km、step_goal、pomo_secs、pomo_mode、pomo_state、pomo_count、pitch、roll、is_level、battery_v、battery_pct、time_synced、day_index、uptime_s、accel_ok、step_signal
/api/history	GET	七天的 day、steps、pomo (最後一筆是今天)
/api/pomodoro	POST	action=toggle reset skip
/api/settings	GET / POST	work_mins、break_mins、stride_cm、weight_kg、step_goal、k_thresh、min_interval、max_interval、confirm_steps、web_pass、sta_ssid、sta_pass
/api/reset_steps	POST	今日步數歸零
/api/time	POST	epoch=秒、tz_min=時區分鐘

序列埠 (115200) 也接受兩個指令：`time <epoch> [tz_min]` 用電腦時間同步 (`tools/sync_time.py` 會自動送)、`status` 印出狀態 JSON。接 USB 開發時不必連 Wi-Fi 就能對時。

C 指令

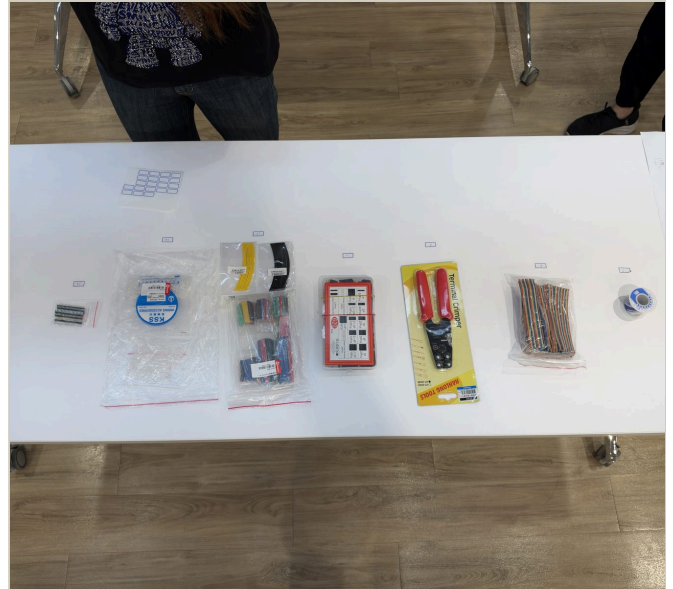
```
make sim-build          # 重編模擬器 (Emscripten)
node tools/qc_sim.mjs    # 16 項自動驗收
make build               # 編真機韌體 (先自動產生字型)
make flash PORT=/dev/cu.usbserial-XXXX # 只接受允許清單中的埠
make monitor PORT=...    # 看序列埠 115200
```

E 教學材料包（依原作 BOM 整理，一組一套）

編號	品項	用途	備註
1	LILYGO TTGO T-Display (ESP32)	主控 + 螢幕 + 充電	附排針，不必焊排針，直接焊線即可
2	GY-291 ADXL345 三軸加速度模組	計步、水平儀	只焊 VCC、GND、SDA、SCL
3	無源蜂鳴器（黑色圓殼）	提示音、旋律	先用 3.3 V 直流測：會響的是有源，退回換貨
4	603450 鋰電池 1000 mAh (JST 1.25 mm)	電源	避免尖物、不放電到 3.3 V 以下
5	KCD1 船型開關	電源開關	串在電池正極
6	焊錫 (0.8 mm)	焊接	含松香芯
7	彩虹排線	感測器與蜂鳴器接線	剪四芯一段、兩芯一段
8、9	端子壓線鉗、杜邦母座與端子	想做可拆接頭時用	原作是直接焊，可省略
10	杜邦接頭套件	同上	
11	熱縮套管套件包	包覆開關與電池焊點	必要，防短路
12	PC 塑膠圓頭十字螺絲 M3×10 (每人 5 顆)	鎖外殼	
13	排針 1×20	備用	T-Display 已附
—	電烙鐵、烙鐵架、墊子	每組一套	手寫補充

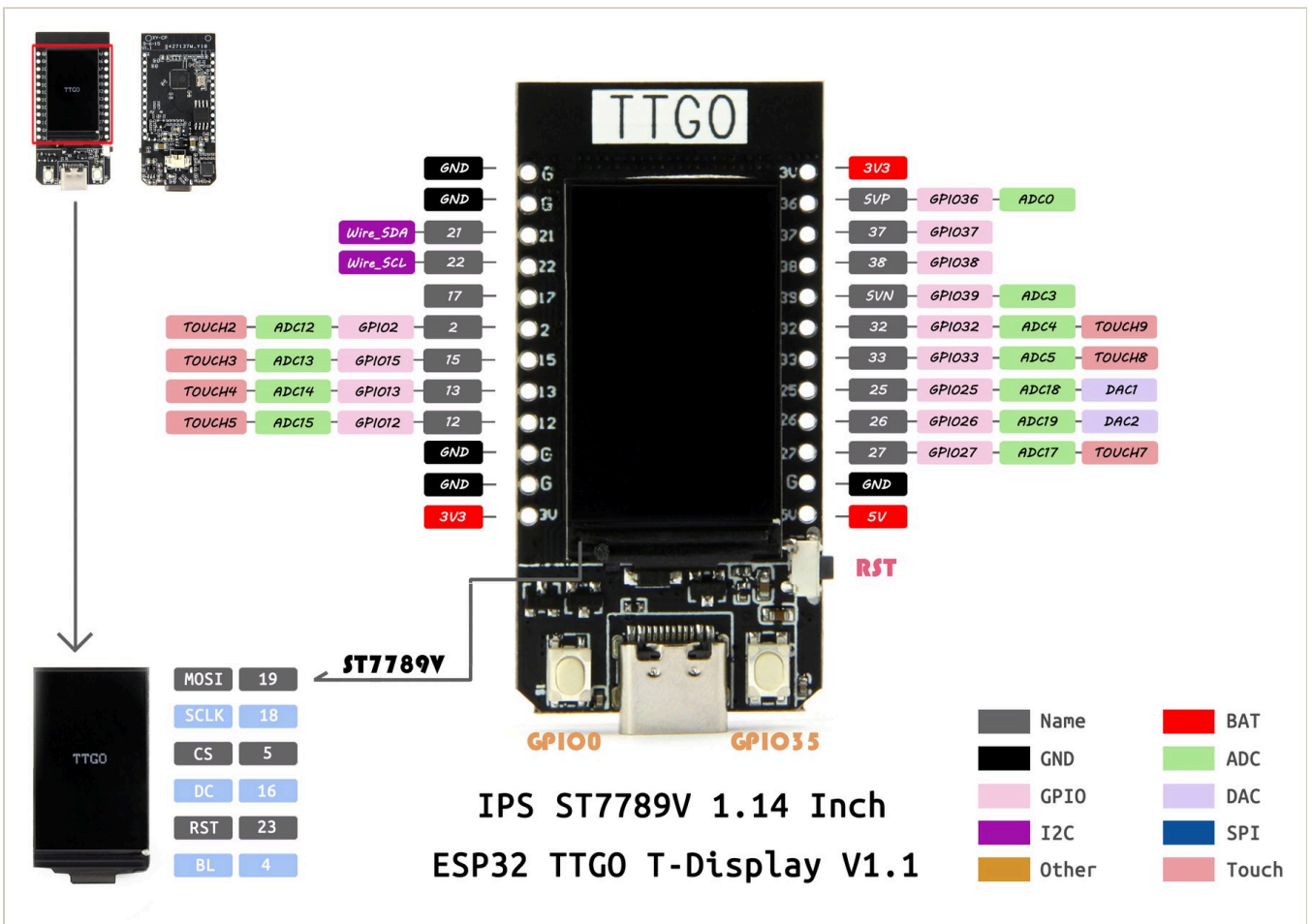


每組的主要零件（編號 1~5）+ 排線與 3D 列印外殼。



共用工具與耗材（編號 6~13）：焊錫、排線、壓線鉗、杜邦套件、熱縮管、螺絲、排針。

F 官方資料與資料手冊



TTGO T-Display 官方腳位圖（LilyGO，MIT 授權，取自官方 GitHub）。電池量測用的 GPIO34（ADC_IN）與其開關 GPIO14（ADC_EN）是板內用途，腳位圖上 GPIO14 只標成 TOUCH4 / ADC14；ADC_EN 的用法要看官方範例程式與電路圖（docs/datasheet/）。

資料	來源	用途
T-Display 腳位圖、電路圖 (ESP32-TFT(6-26).pdf)、TFT_eSPI 設定檔	github.com/Xinyuan-LilyGO/TTGO-T-Display (MIT) ; 電路圖副本在專案 docs/datasheet/	查腳位、背光與分壓電路、充電晶片
ESP32 Series Datasheet	espressif.com → Documentation → esp32_datasheet_en.pdf	GPIO 限制 (34~39 只能輸入、無上拉)、開機模式腳、耗電模式
Neil Zhao, “Full-Featured Pedometer Design Realized with 3-Axis Digital Accelerometer”, <i>Analog Dialogue</i> 44-06, 2010	analog.com → Analog Dialogue	計步演算法的經典參考：峰值偵測、時間窗、連續步伐判定、步幅估算
ADXL345 Data Sheet	analog.com → ADXL345 → Data Sheet	暫存器 (0x2C BW_RATE、0x2D POWER_CTL、0x31 DATA_FORMAT、0x32 DATA0)、3.9 mg/LSB、I2C 位址 0x53 / 0x1D
MPU-6000/6050 Register Map	invensense.tdk.com → MPU-6050 → Register Map	0x6B PWR_MGMT_1、0x1C ACCEL_CONFIG、0x3B ACCEL_XOUT_H、WHO_AM_I 0x75
ST7789V Datasheet	Sitronix (各代理商網站可下載)	135×240 的位移 (offset 52 / 40)、Sleep In / Out 指令
TP4054 鋰電池充電 IC	各廠 datasheet (TopPower 等)	充電電流由板上電阻決定、充飽 4.2 V

資料手冊是「原廠說它會怎樣」，和說明書一樣要用實測對照：本講義 GPIO14 那一條就是查腳位圖才發現的。

G 硬體除錯流程

症狀	依序檢查
螢幕完全不亮	① 船型開關開了嗎？② 三用電表量電池：低於 3.3 V 先充電。③ 拔掉所有外接線、只插 USB：亮就是外接短路，不亮才是板子壞。
螢幕寫「找不到加速度計」	① GY-291 的 VCC / GND 接反或鬆脫？② SDA→21、SCL→22 有沒有對調？③ CS 腳是不是碰到 GND (會切成 SPI)？④ 燒一支 I2C 掃描程式看掃不掃得到 0x53。模擬器「接線」分頁可先演練 ①~③。

蜂鳴器沙沙聲或很小聲	① 用 3.3 V 直流短觸：連續嗶＝有源（拿錯了）；只「喀」一下＝無源（正確）。② 負極有沒有接到 GND？③ 是不是接到 GPIO25 以外的腳？
電量一直 0% 或亂跳	① 讀 GPIO34 前有沒有把 GPIO14 拉高？② 公式有沒有 $\times 2$ ？③ 用電表量電池實際電壓對照。
燒錄失敗 Failed to connect	按住左鍵（GPIO0）→ 按一下 RST → 放開左鍵，再燒；USB-C 對 USB-C 線可能抓不到，換 USB-A 線。
模擬器正常、真機不正常	這就是模擬範圍之外的事：量電壓、看序列埠開機訊息、換線、換模組，一次只改一個變數。

H 審稿紀錄：另一版修訂稿的查證結果

2026-10-06 收到另一個 AI 產出的「修訂增補版」講義。逐條查證後，正確且有價值的已併入本版，錯誤的列於下表供對照（依本專案「修正版本說明格式」慣例）。

修訂稿的說法	查證	採用	修正後內容 / 理由
鋰電池安全守則（不同時剪兩線、熱縮管、3.0 V 下限）	正確	採用	第 2 節新增安全守則框
GPIO14 ADC_EN： 100k:100k 分壓、約 21 μ A 漏電、量完拉低	正確	採用	第 3.4 節；韌體同步改成量完拉低
活動消耗 vs 基礎代謝率的邊界	正確	採用	第 4.4 節框，補 Mifflin—St Jeor 例子
硬體除錯流程圖	正確	採用	附錄 G，改成表格以利列印
BMI270、ICM-42688-P 選型列	部分	修正後採用	ICM-42688-P 加速度雜訊是 70 μ g/ $\sqrt{\text{Hz}}$ (datasheet)，不是 0.28 μ g/ $\sqrt{\text{Hz}}$ （差 250 倍）
ESP32 高電位 >2.64 V、低電位 <0.66 V	錯誤	修正	datasheet 是 0.75 $\times$ VDD ($\approx$ 2.5 V) 與 0.25 $\times$ VDD ($\approx$ 0.8 V)
$\alpha=0.3$ 約等於截止 3~4 Hz； $\beta=0.02$ 時間常數「數秒級」	錯誤	修正	$f_c = -\ln(1-\alpha)/(2\pi\Delta t)$ ： $\alpha=0.3 \rightarrow 2.8 \text{ Hz}$ ； $\beta=0.02 \rightarrow 0.16 \text{ Hz}$ 、時間常數約 1 秒
演算法「參考 ADI AN-900」	編號有誤	修正	Neil Zhao 的文章是 <i>Analog Dialogue</i> 44-06 (2010) 「Full-Featured Pedometer Design Realized with 3-Axis Digital Accelerometer」；ADI 的計步應用筆記是 AN-602 與 AN-2554。本演算法與該文同屬「峰值+時間窗+連續確認」一族，但非逐條實作

水平儀「增加中位數平滑濾波與動態水平校正」	與實作不符	不採用	本韌體水平儀用指數平滑（係數 0.2），沒有中位數濾波；歸零功能在「量角器」卡帶，不在水平儀
GY-291 上拉電阻 4.7 kΩ	未逐一查證	保留為「約」	常見模組為 4.7 kΩ，但各批次不同；講義只寫「板上有上拉電阻」
Pico 是「Cortex-M0+、Thumb-2」	部分	修正	Cortex-M0+ 用 ARMv6-M，主要是 Thumb 指令集（只含少數 Thumb-2 指令）；講義寫「ARM Thumb」
刪掉實拍照片、材料包、第 10 節遊戲化	—	不採用	修訂稿是純文字版，缺本版的照片與後續內容；以本版為主幹合併

D 圖片出處

- 封面、原作內部、T-Display 接線、ADXL345 特寫、零件、材料包、說明書：作者實拍（2026 年 10 月）。
- MEMS 原理示意、走路情境：AI 生成（FLUX.2 [klein] 4B，本機生成，Apache-2.0），為示意圖，非實物；中文說明為本文後製。
- LIS3DH 模組照片：SparkFun Electronics，CC BY 2.0，取自 Wikimedia Commons。
- GY-521 MPU-6050 模組照片：Nevit Dilmen，CC BY-SA 3.0，Wikimedia Commons。
- Raspberry Pi Pico：Phiarc，CC BY-SA 4.0，Wikimedia Commons。
- T-Display 腳位圖：Xinyuan-LilyGO，MIT License，github.com/Xinyuan-LilyGO/TTGO-T-Display。
- QR Code 編碼：Project Nayuki「QR Code generator library」（C 版），MIT License，nayuki.io；程式在 `firmware/app/third_party/`。
- 接線圖、架構圖、選型流程、逆向對照、計步訊號圖、模擬器與儀表板畫面：本文繪製 / 擷取。原作英文儀表板：學生作品，取自其韌體內嵌網頁，僅供教學比較。

本講義依著作權法第 46 條為學校授課需要重製他人著作之必要範圍內使用，各圖依授權條款標示來源。原作「GOTT Pedometer」為學生專題作品，本講義之新版韌體為另行設計，非原作之重製。